



KCED

Learning Augmented

24ADI003-MACHINE LEARNING

UNIT 2: **REGRESSION MODELS**

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

- **What Is Learning in Regression?**
- In **regression**, the goal is simple:
- **Learn a function that maps inputs (X) to a continuous output (Y)**

Example:

- X = hours studied
- Y = exam marks
- We assume:

$$Y = f(X) + error$$

- The challenge is:
- We **do not know** the true function f
- We must **approximate it from data**
- This approximation can fail in **two opposite ways**:
- Model learns **too little** → **Underfitting**
- Model learns **too much** → **Overfitting**

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING (MACHINE LEARNING TOO LITTLE)

- Imagine teaching a child the concept of **temperature** using only:
- “Hot” and “Cold”
- But the real world has:
- Warm, cool, freezing, boiling, etc.
- → The explanation is **too simple** to describe reality.
- That is **underfitting**.

Formal Definition

- **A hypothesis underfits the data if it cannot capture the underlying relationship between input and output variables.**

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING (LEARNING TOO LITTLE)

Why Underfitting Happens (Step by Step)

- Underfitting occurs when:
- The model is **too simple**
 - Linear model for nonlinear data
- Important variables are ignored
- Training is stopped too early
- Regularization is too strong

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

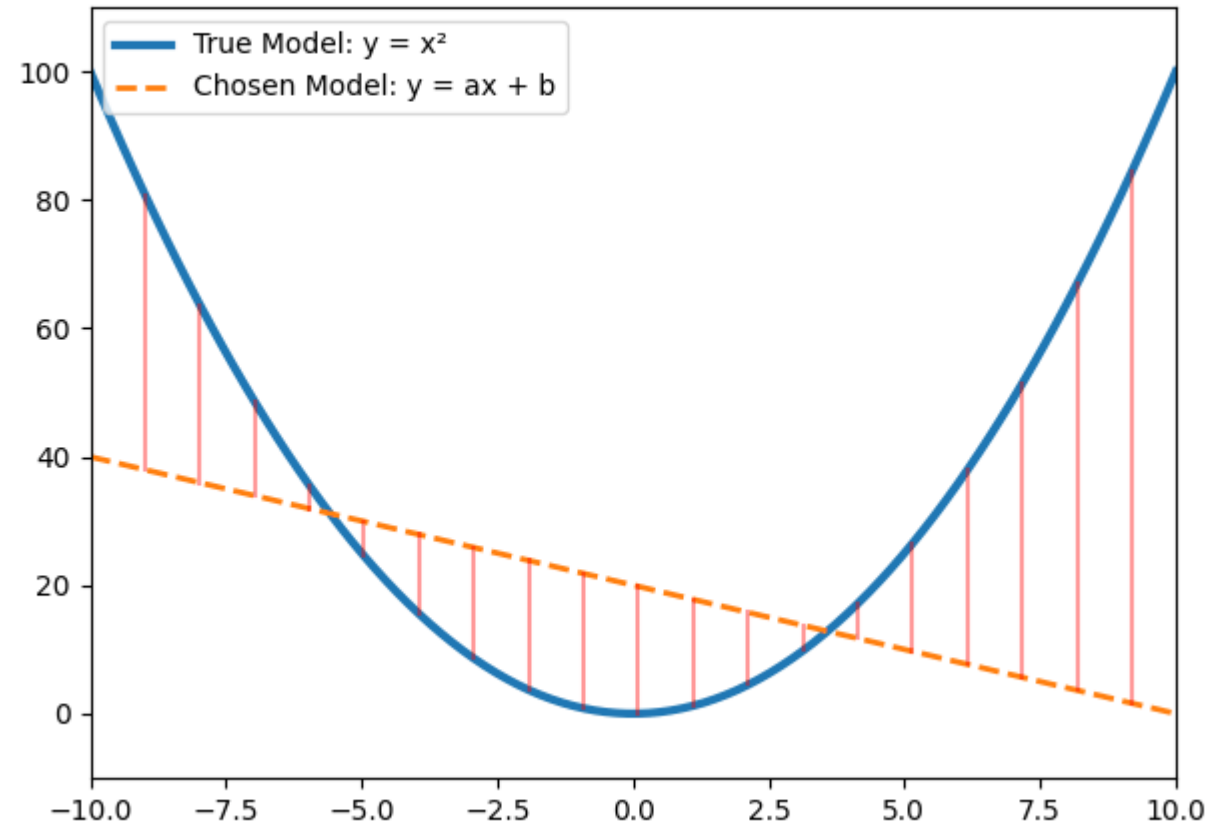
UNDERFITTING :Mathematical View

No matter how we choose a and b ,
a straight line **cannot represent a curve.**

- The error remains large
- This is **high bias**

**High bias occurs
when the model is
too simple to
capture the true
pattern, no matter
how well it is
trained.**

Trying Linear Model: $y = -2.00x + 20.00$
Error remains large → **HIGH BIAS**



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :Mathematical View

Example

Given Dataset

x	y
1	1
2	4
3	9

The output increases **non-linearly**
The rate of increase itself is increasing

Squared Errors

We square errors to:

- Remove negative sign
- Penalize larger deviations

$$(1 - 2)^2 = 1$$

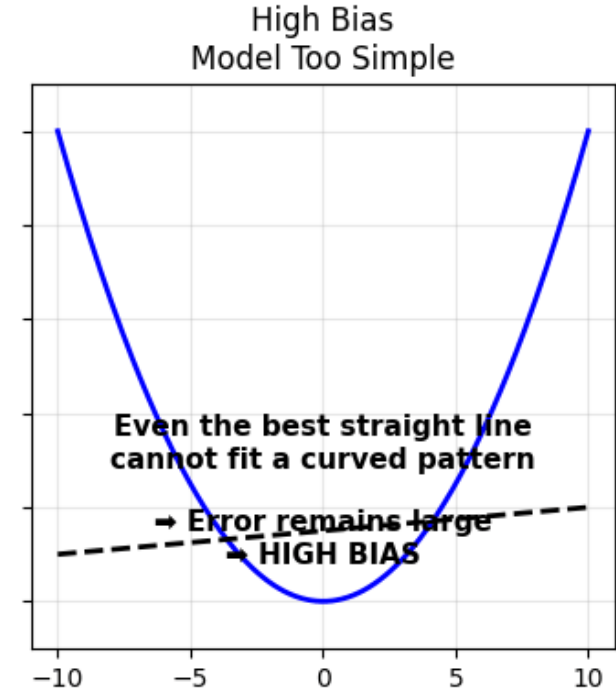
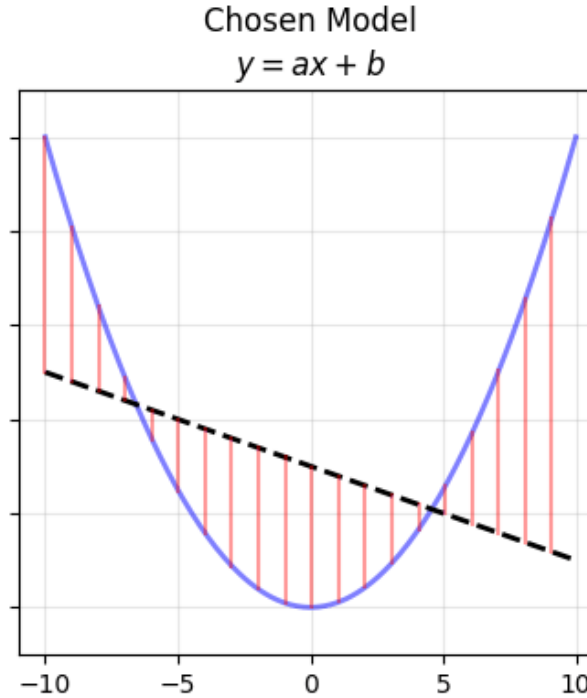
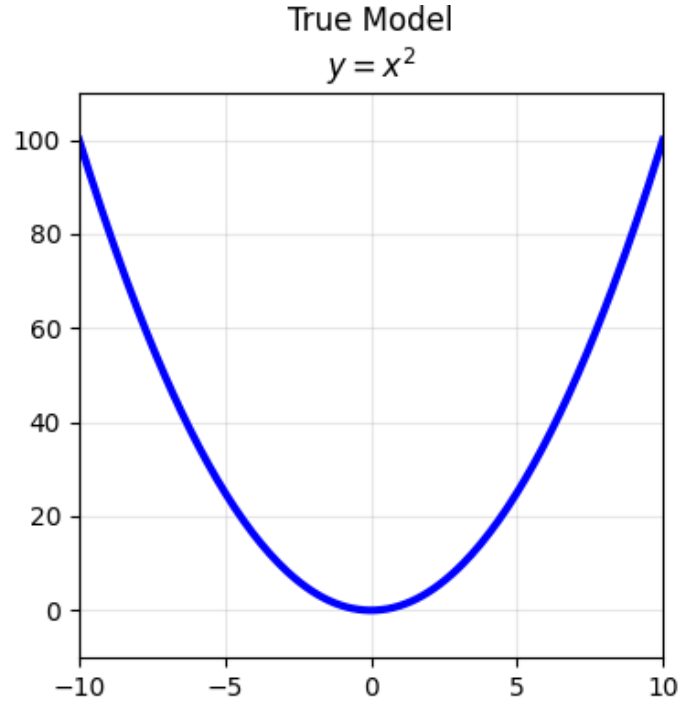
$$(4 - 4)^2 = 0$$

$$(9 - 6)^2 = 9$$

x	Actual y	Predicted $y = 2x$	Error
1	1	2	1
2	4	4	0
3	9	6	3

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :Mathematical View



The errors are **not random**.

- Error at $x=1 \rightarrow$ model **overestimates**
 - Error at $x=3 \rightarrow$ model **underestimates**
 - Error **increases with x**
- $\rightarrow$ This indicates a **systematic failure**, not noise.

Why This Is Underfitting

- The model cannot represent curvature
- Error remains large even after training
- Adding more data will **not fix** the problem

Core idea:

The hypothesis space is too small.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :Mathematical View

Residual=Actual value–Predicted value

A pattern in residuals means the model is making the *same kind of mistake again and again*.

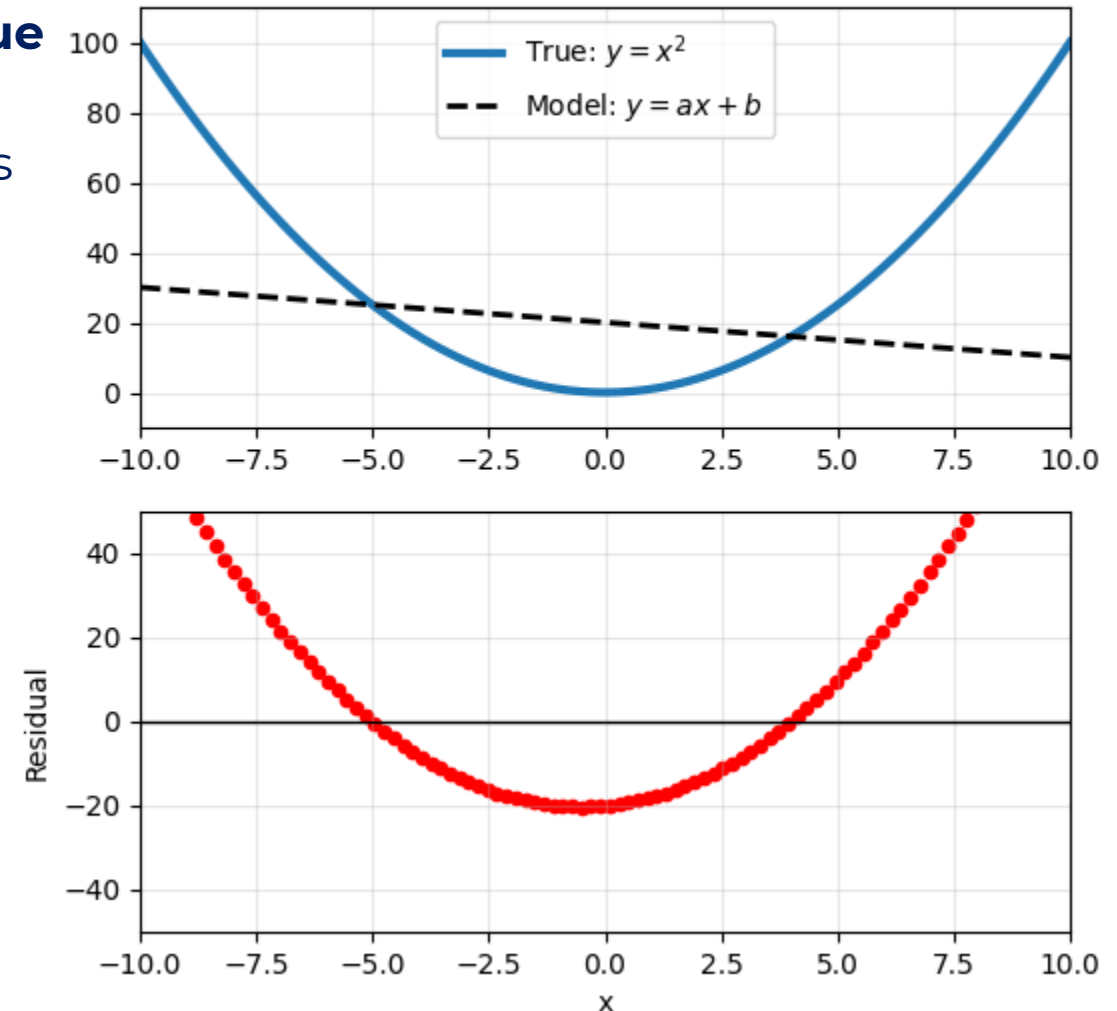
Then conclude:

“The model assumption is wrong.”

- The model is **too simple**
- It cannot capture curvature
- This is **UNDERFITTING**

- Residuals are **positive at the ends**
- Residuals are **negative in the middle**
- The points form a **U-shape**

Underfitting: Residuals show a clear pattern
→ Errors are systematic → **HIGH BIAS**



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

Symptoms of Underfitting (Deep Explanation)

• 1. High Training Error

- Even on data the model has already seen:
- Predictions are poor
- Loss function remains high
- → Model never learned the pattern.

2. High Testing Error

- On unseen data:
- Errors remain large
- Performance does not improve
- → Indicates **high bias**, not variance.

3. Poor Predictions Everywhere

Small $x \rightarrow$ incorrect

Medium $x \rightarrow$ partially correct

Large $x \rightarrow$ very incorrect

→ Errors grow consistently.

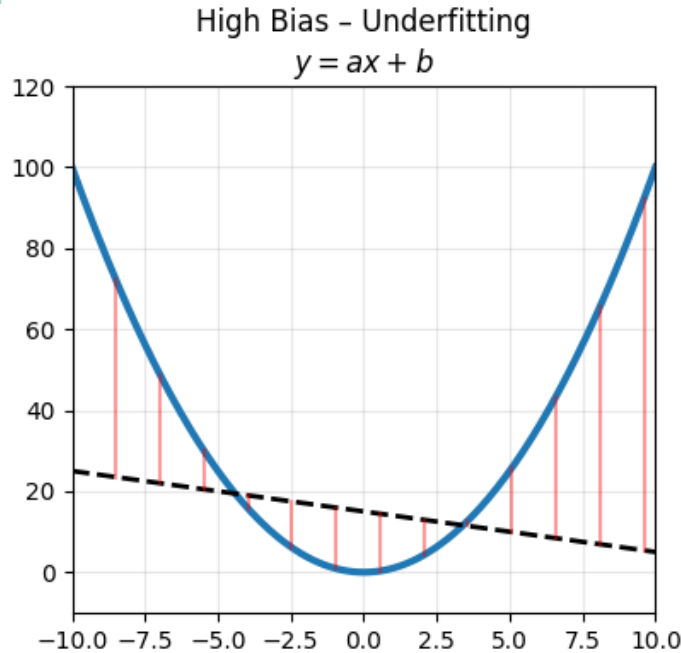


UNDERFITTING :

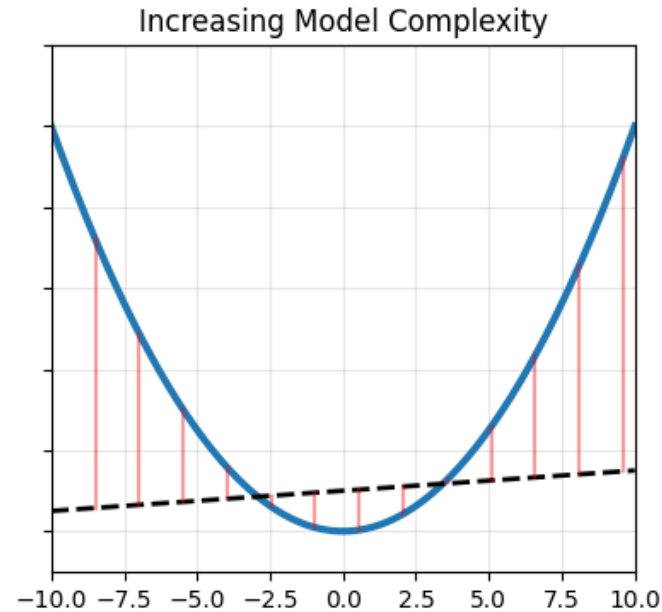
How to Fix Underfitting

1. Increase Model Complexity

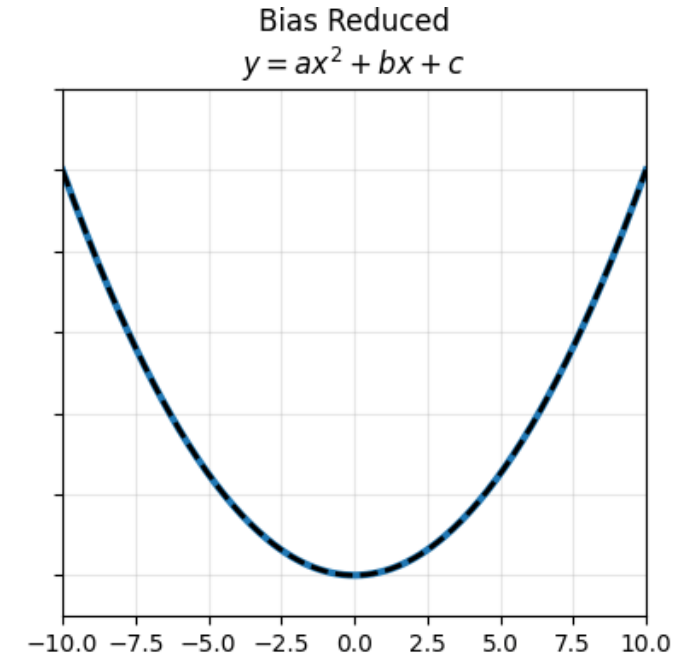
Increasing model complexity enables the model to capture nonlinear structure, reducing systematic residuals and high bias



“Straight line → cannot bend → systematic residuals → high bias.”



Model becomes more flexible → errors reduce



Correct complexity → residuals disappear → bias reduced.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

2. Add Meaningful Features

Underfitting often occurs because:

- Important information is missing

Example:

Predicting temperature using:

- Only time of day

Better features:

- Depth
- Salinity
- Location

→ More information → better learning

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

3. Reduce Regularization

What Does Regularization Do?

- Regularization adds a **penalty** to the learning process.
- The model is told:
- “Do not use very large coefficients. Keep them small.”

This is useful because:

- Large coefficients make the model sensitive to noise
- Small coefficients make the model smoother and more stable

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

3. Reduce Regularization

What Happens When Regularization Is Too Strong?

- Regularization strength is controlled by λ (**lambda**).

When λ is very large:

- The penalty dominates the cost function
- The model prioritizes **small coefficients** over data fit
- Mathematically:

$$\text{Cost} = \text{Error} + \lambda \times \text{Penalty}$$

- If λ is large:

$$\lambda \times \text{Penalty} \gg \text{Error}$$

Effect on Coefficients

- Coefficients shrink toward **zero**
- Some coefficients may become **almost zero**
- Model loses flexibility
- → The model starts behaving like a **very simple model**

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

3. Reduce Regularization

Why Strong Regularization Causes Underfitting

- Consider polynomial regression:

$$y = a_0 + a_1x + a_2x^2$$

- With strong regularization:

$$a_2 \approx 0$$

- Model becomes:

$$y \approx a_0 + a_1x$$

- The curve collapses into a straight line
- Curvature disappears
- Underfitting occurs

LEARNING IS
AN AUGMENTED
PROCESS

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

3. Reduce Regularization

Why Reducing λ Helps

When λ is reduced:

Penalty term becomes weaker

Model is allowed to:

- Use larger coefficients

- Represent complex patterns

- Fit curvature and interactions

→ **Patterns hidden in data begin to emerge**

Example

Predicting marks using:

- Hours studied
- Revision time
- Sleep hours

If λ is too large:

- All coefficients become small
- Model predicts nearly the same mark for everyone

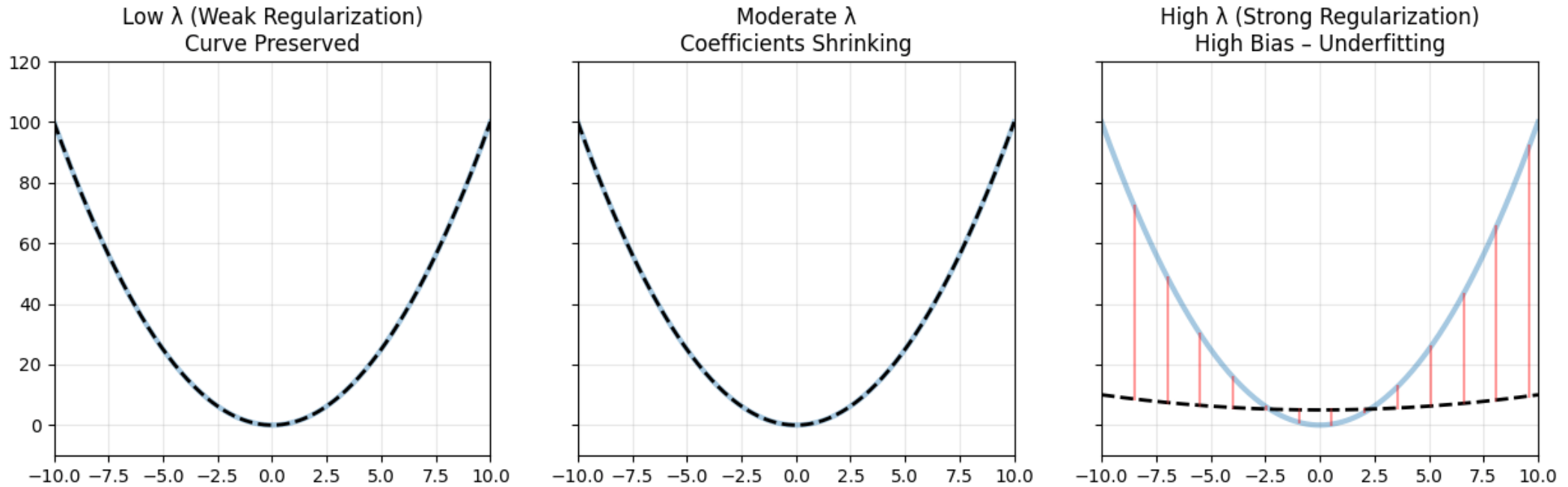
Reducing λ :

- Allows study hours to dominate
- Meaningful influence is restored

Excessive regularization increases bias and can lead to underfitting.

UNDERFITTING :

How to Fix Underfitting



Regularization fights variance, but too much regularization creates bias.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

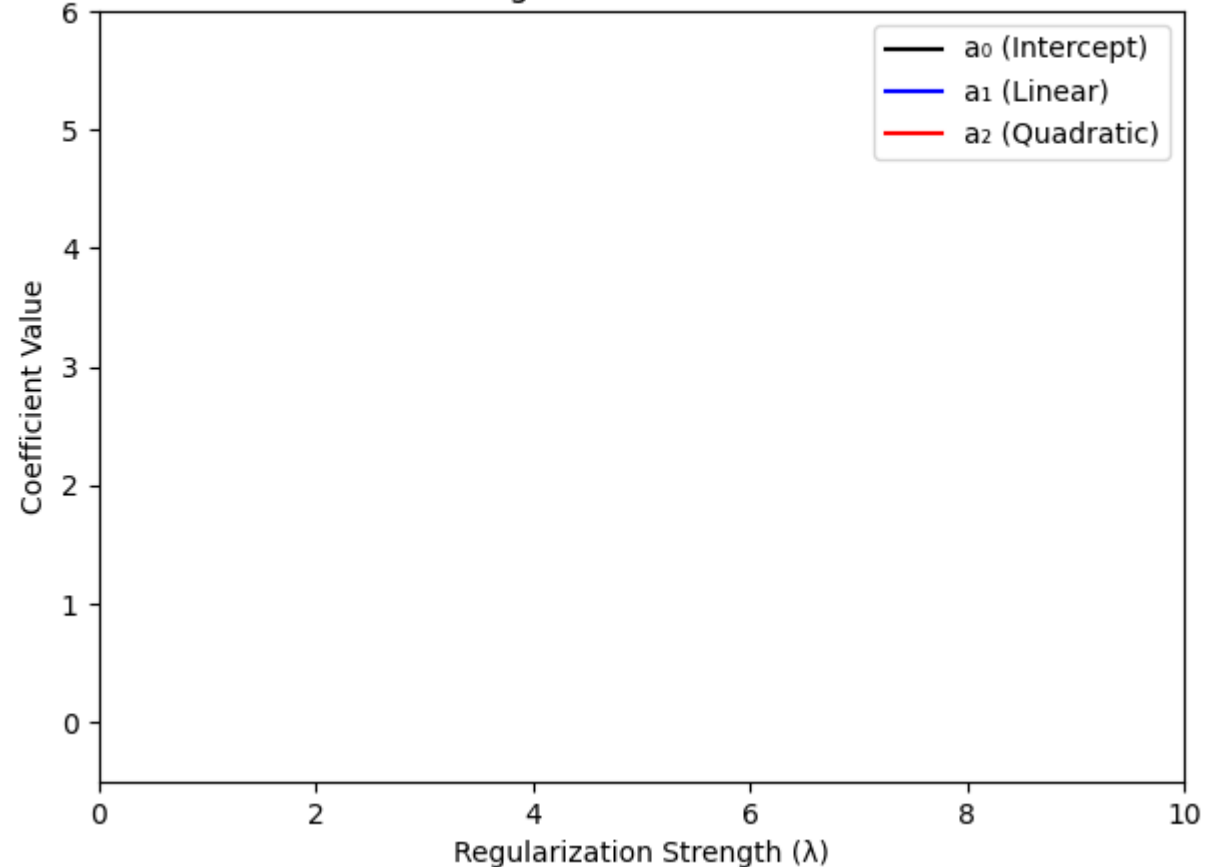
How to Fix Underfitting

“Regularization pulls coefficients toward zero.”

- Small $\lambda \rightarrow$ model is free
- Large $\lambda \rightarrow$ model is restricted
- Too large $\lambda \rightarrow$ model becomes too simple
→ **Curvature disappears → underfitting**

- $\lambda = 0 \rightarrow$ No regularization $\rightarrow$ complex model
- $\lambda \uparrow \rightarrow$ Coefficients shrink
- λ **very large** $\rightarrow$ Model almost flat $\rightarrow$ underfitting

Effect of Regularization on Coefficients



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

4. Train Longer / Better Optimization

Why Training Time Matters

Most regression models learn parameters using **gradient descent**.

Gradient descent:

Starts from an initial guess

Improves parameters step by step

Learning does **not happen instantly**.

Early Stopping

If training is stopped too early:

Gradient descent has not reached the minimum

Parameters are still far from optimal

Error remains high

→ Model appears underfit

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

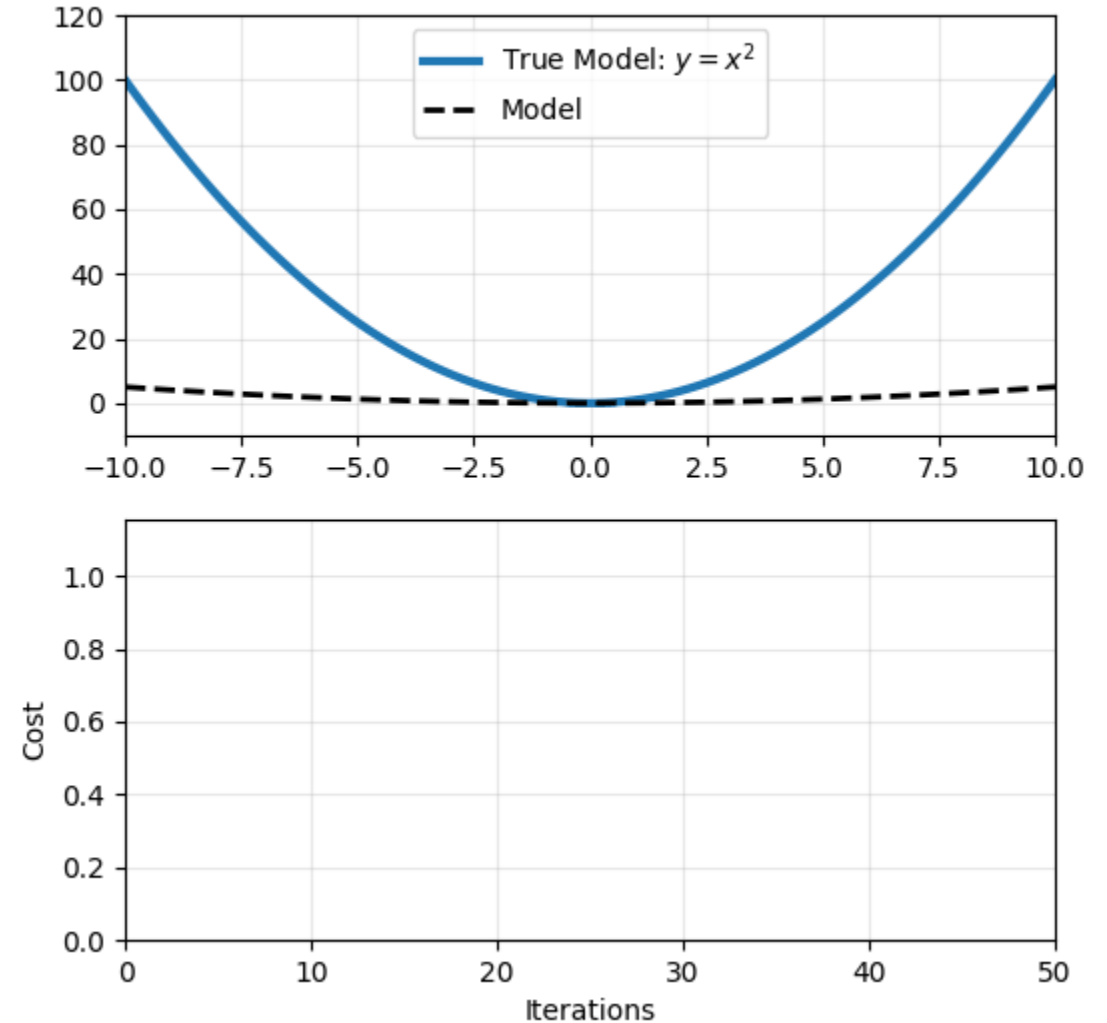
How to Fix Underfitting

4. Train Longer / Better Optimization

Because learning stopped early,
the model never learned the full curve

Early stopping limits training time.
Too little training = parameters remain
suboptimal = high bias.

Training in Progress: Model Learning



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

Training loss vs iterations with **model complexity evolving over time.**

Early Stopping → Underfitting

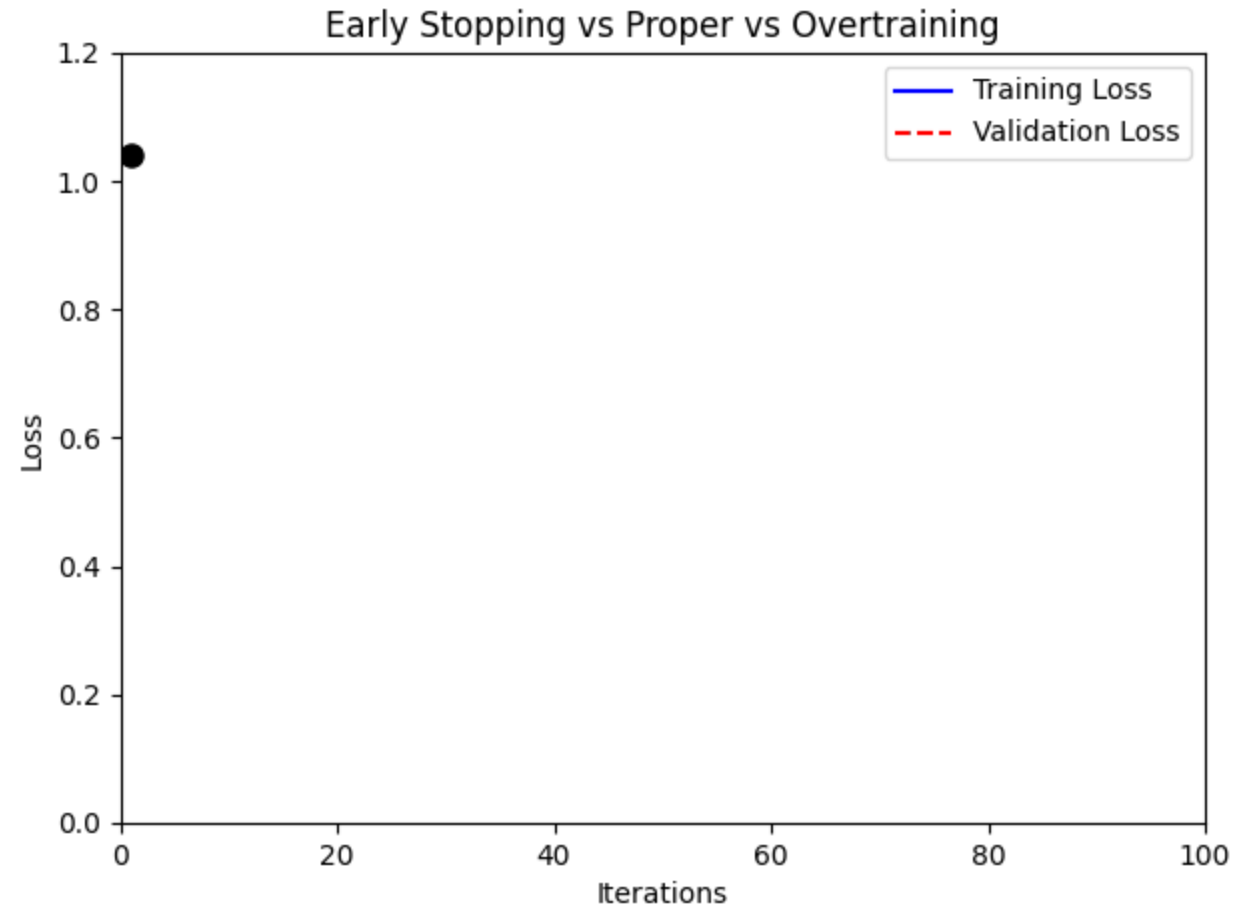
- Training stopped too soon
- Model hasn't reached minimum
- High bias

Proper Stopping → Best Generalization

- Stop near the valley bottom
- Balanced bias–variance

Overtraining → Overfitting

- Training continues too long
- Model memorizes noise
- High variance



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

4. Train Longer / Better Optimization Increasing Epochs (More Learning Time)

An **epoch** = one full pass through the training dataset.

Increasing epochs:

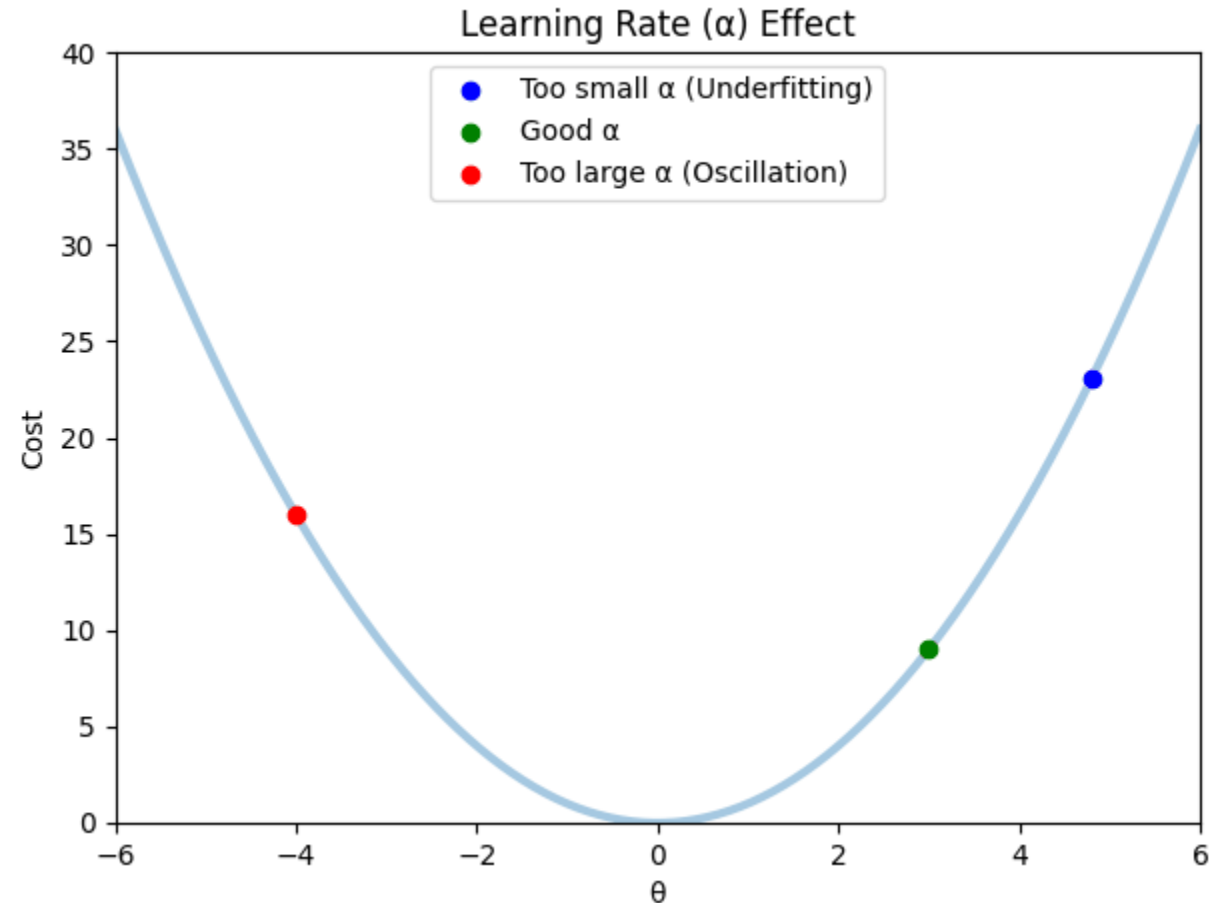
- Allows parameters to adjust further
- Error keeps decreasing
- Model reaches better fit

Learning Rate (α) and Underfitting If α is too small:

- Learning is extremely slow
- Model appears undertrained
- Training stops before progress is visible

If α is too large:

- Model overshoots the minimum
- Training becomes unstable



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

Example

Initial:

$\theta = 0$

True $\theta \approx 5$

With **few epochs:**

θ reaches only 1.2

Model still weak

With more epochs:

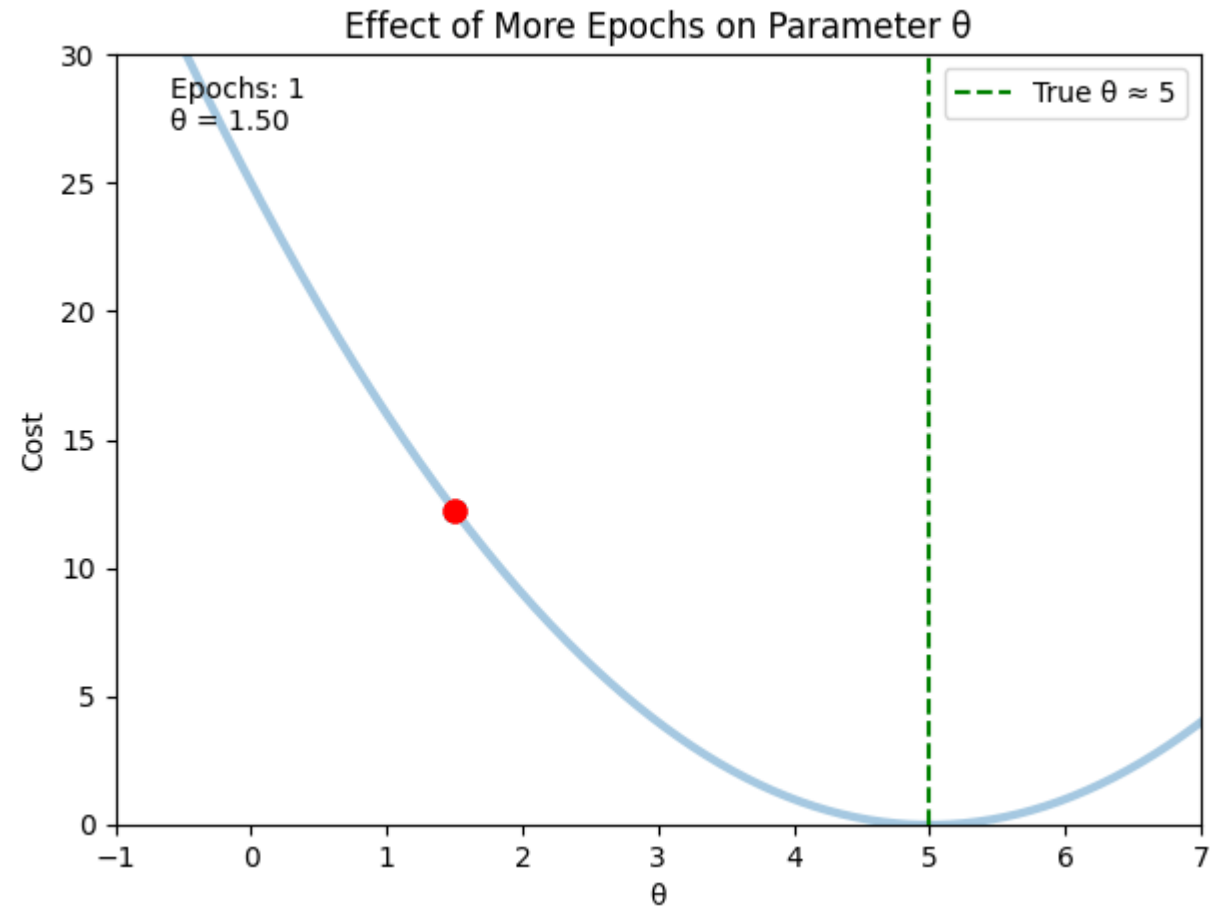
$\theta \rightarrow 4.8$

Error minimized

If trained only for:

5 epochs $\rightarrow$ model underfits

100 epochs $\rightarrow$ model converges



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

UNDERFITTING :

How to Fix Underfitting

Issue	Effect	Solution
λ too large	Coefficients shrink	Reduce λ
Early stopping	Model not learned	Increase epochs
α too small	Slow learning	Increase α slightly
α too large	Instability	Reduce α

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING (MODEL LEARNING TOO MUCH)

Imagine a student who:

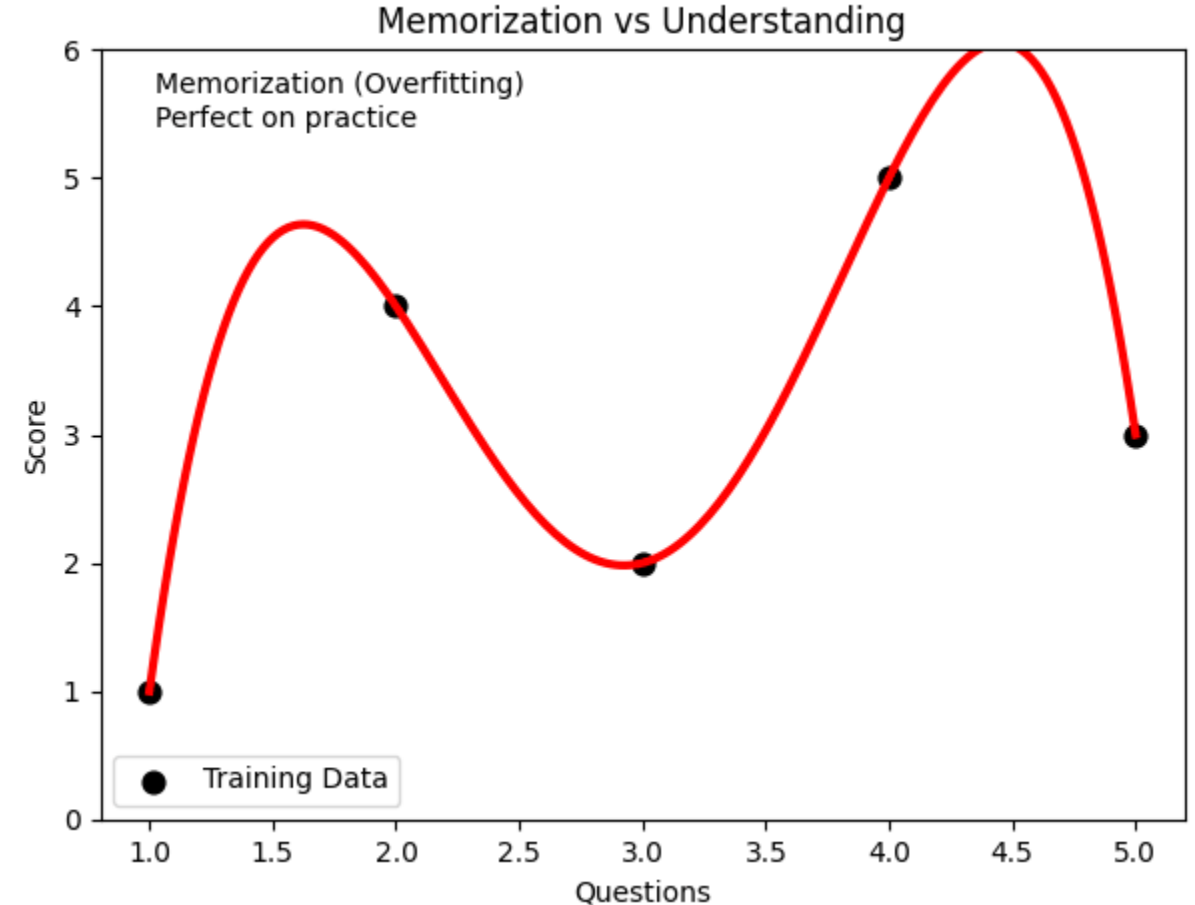
- Studies **only previous question papers**
- Memorizes answers word-by-word
- Scores full marks in practice tests
- Fails badly when questions change slightly

The student has **memorized**, not **understood**.

This is exactly what an **overfitted model** does.

→ It learns **training data perfectly**

→ It fails on **new, unseen data**



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

Overfitting occurs when a model captures noise and random fluctuations in training data instead of the underlying pattern, resulting in poor generalization performance.

Why Overfitting Happens

1. Too Many Parameters

Model becomes excessively flexible

Can fit even meaningless patterns

Example:

10 data points

50 parameters

→ Infinite models can fit the data

2. Small Dataset

Model has insufficient examples

Noise looks like real signal

3. Noise Is Mistaken as Signal

Noise = random error or disturbance

Example:

Measurement errors

Sensor inaccuracies

Human errors

Overfitted models **chase noise**.

4. No Regularization

No penalty for large coefficients

Parameters grow uncontrollably

LEARNING AUGMENTED

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

Mathematical Insight

High-degree polynomial:

$$y = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots$$

With enough terms:

Curve twists sharply

Passes through **every training point**

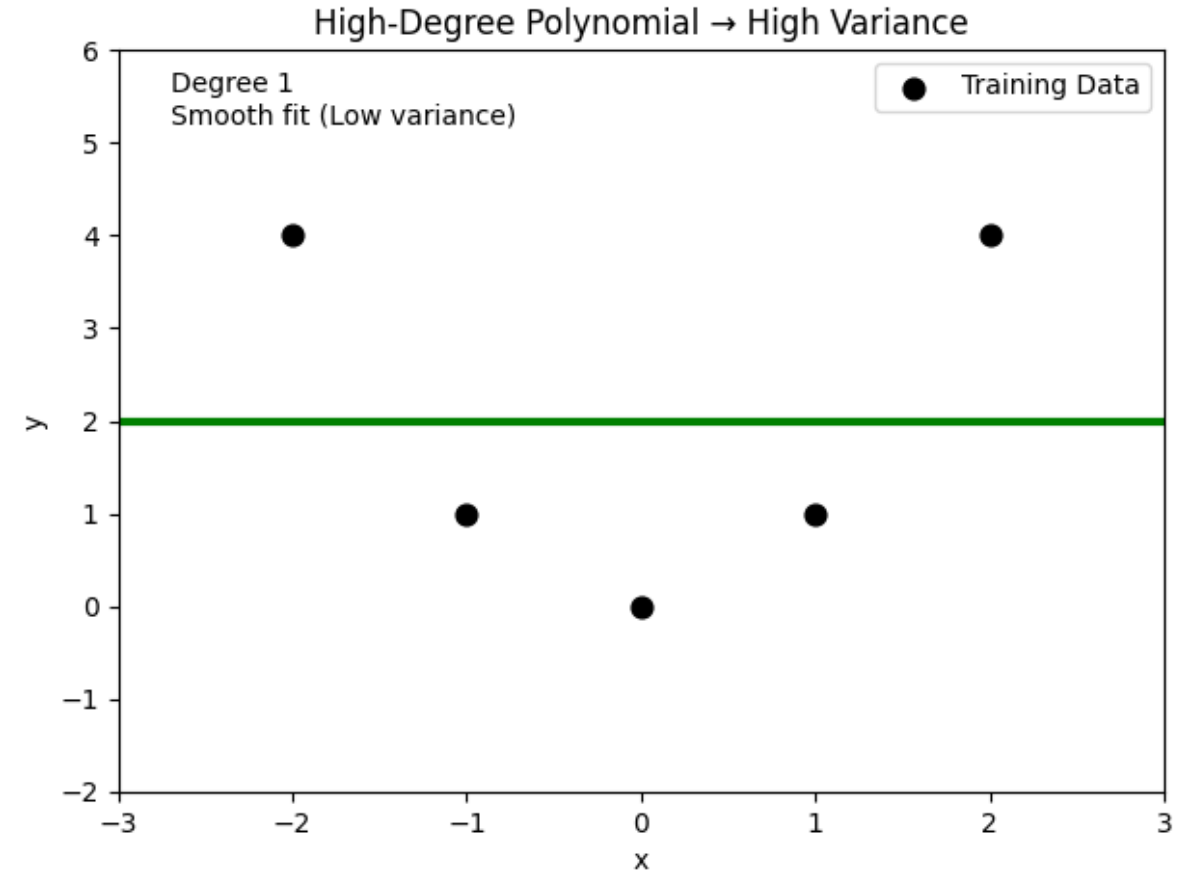
Training error → **zero**

But:

Curve oscillates wildly

Predictions outside training range are unreliable

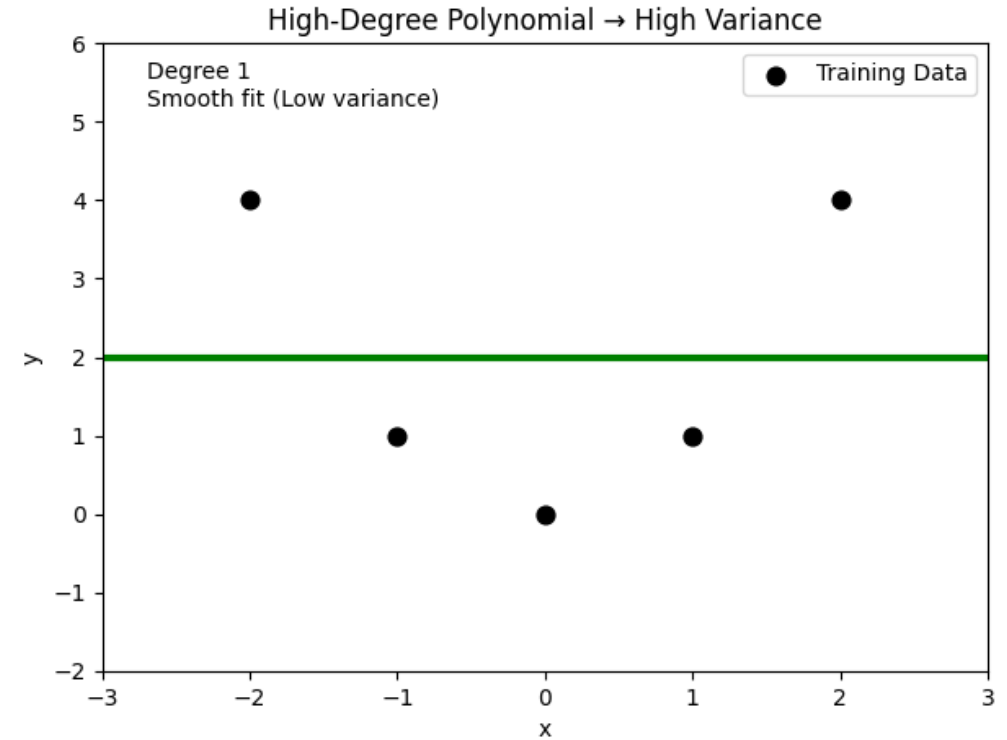
→ **High variance model**



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

Mathematical Insight



Degree	Training Error	Behaviour	Variance
1-2	Moderate	Smooth	Low
3-5	Low	Wavy	Medium
7+	~ 0	Oscillatory	High

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

Example: Given Data

x	y
1	2
2	4
3	6

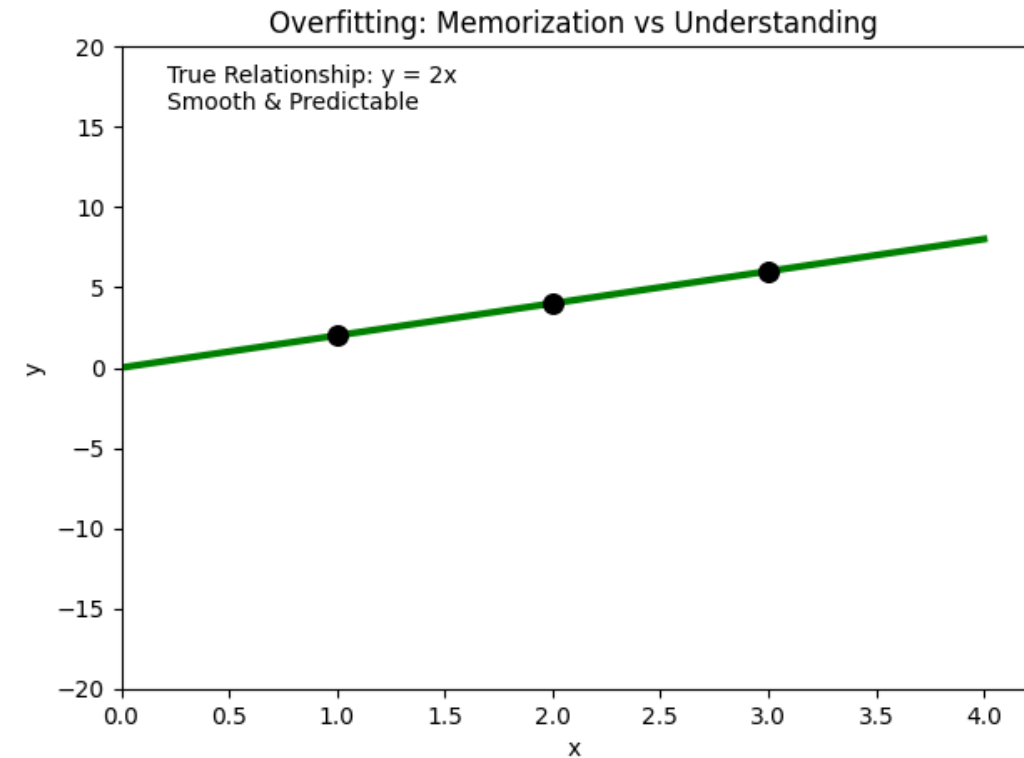
$$y = 2x$$

This green line understands the rule.

This red curve memorizes three answers.

The moment I ask a new question, it fails.
Pause.

“Zero training error does NOT mean intelligence.”



Overfitting occurs when a model fits training data perfectly but produces unstable predictions for unseen inputs due to high variance.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

Symptoms of Overfitting

- Very low training error
- Very high testing error
- Highly fluctuating predictions
- Sensitive to small data changes

LEARNING
AUGMENTED

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

Overfitting occurs when a model **memorizes training data** instead of learning the **true underlying pattern**. The following methods help a model **generalize better** to unseen data.

1. Collect More Data

Why It Works

Overfitting is often caused by **insufficient training examples**.

When the dataset is small:

Noise appears as meaningful pattern

Model fits random fluctuations

Adding more data:

Reveals the true trend

Reduces influence of noise

Forces model to learn stable patterns

→ **Variance decreases**

Example

Suppose you estimate average height using:

5 students → large error

500 students → accurate estimate

More data = better generalization.

Variance of estimator:

$$\text{Variance} \propto \frac{1}{n}$$

As **n increases**, variance decreases.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

2. Use Regularization

Why It Works

Overfitting happens when:

Model parameters become too large

Model becomes overly sensitive

Regularization:

Adds a penalty for large coefficients

Encourages simpler models

(Mitchell refers to this as **complexity control**)

Instead of minimizing:

Error

We minimize:

$$\text{Error} + \lambda \times \text{Model Complexity}$$

Effect

Large weights discouraged

Noise fitting reduced

Smoother decision boundaries

→ **Lower variance**

• Small $\lambda \rightarrow$ overfitting

• Large $\lambda \rightarrow$ underfitting

• Optimal $\lambda \rightarrow$ best generalization

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

3. Choose a Simpler Model

Complex models:

Have many parameters

Can represent many shapes

Easily fit noise

Simpler models:

Have fewer degrees of freedom

Focus on dominant patterns

Complex Model	Simpler Alternative
10-degree polynomial	Linear / quadratic
Deep neural network	Shallow network
Many features	Selected features

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

4. Use Cross-Validation

Why It Works

Training error alone **cannot detect overfitting**.

Cross-validation:

Estimates performance on unseen data

Helps compare models objectively

Split data into:

Training set

Validation set

Or use **k-fold cross-validation**:

Data split into k parts

Each part used once as validation

Situation	Observation
Good model	Training error $\approx$ Validation error
Overfitting	Training error $\ll$ Validation error

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

5. Early Stopping

During training:

Initially model learns useful patterns

Later starts fitting noise

Early stopping:

Stops training before memorization

Common in:

- Neural networks
- Gradient-based learning

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

OVERFITTING

How to Fix Overfitting

Method	Core Effect	What It Reduces
More data	Improves pattern reliability	Variance
Regularization	Penalizes complexity	Variance
Simpler model	Limits flexibility	Variance
Cross-validation	Detects overfitting	Model selection error
Early stopping	Prevents memorization	Variance

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Training a machine learning model means **learning the best parameter values** so that predictions are as close as possible to the actual outputs.

What Does “Best” Mean?

“Best” parameters are those that:

- Minimize prediction error
- Generalize well to unseen data

This is expressed as a **minimization problem**:

$$\min_{\theta} J(\theta)$$

where:

- $J(\theta)$ is the **cost (loss) function**
- θ are the model parameters

Gradient descent is an iterative optimization algorithm that updates model parameters in the direction of the negative gradient of the cost function to minimize prediction error efficiently for large-scale machine learning problems.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Why Not Solve Directly?

Some models (like simple linear regression) have **closed-form solutions**:

$$\theta = (X^T X)^{-1} X^T y$$

However, these solutions:

Require matrix inversion (computationally expensive)

Fail when:

- Dataset is very large

- Number of features is huge

- Model is nonlinear or deep

→ **Not practical for modern ML**

Why Gradient Descent Is Preferred

Gradient descent:

- Avoids matrix inversion

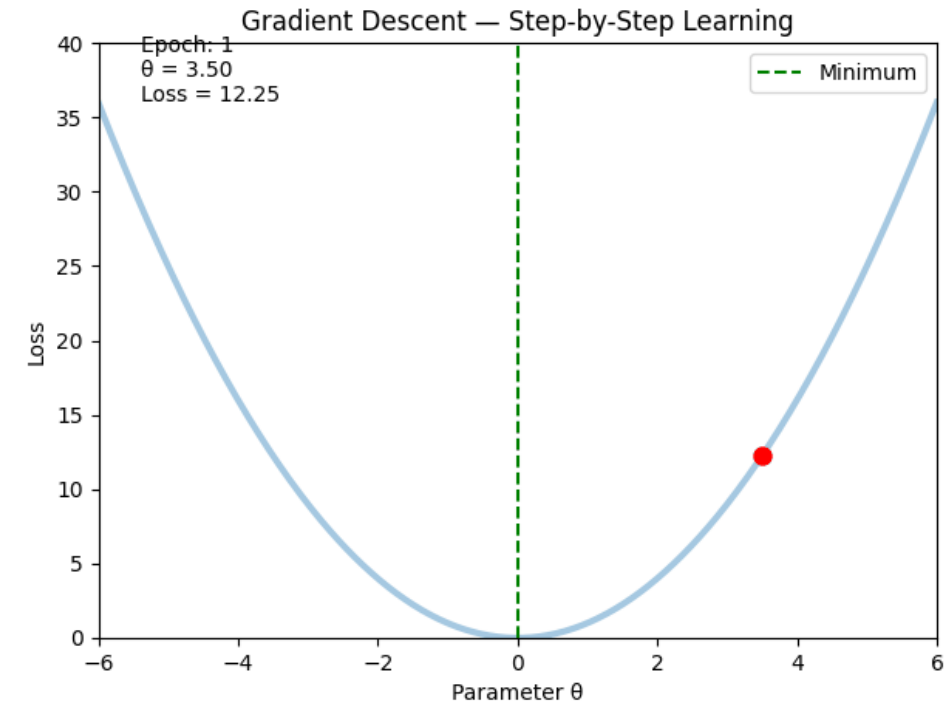
- Updates parameters iteratively

- Scales efficiently to:

- Millions of samples

- High-dimensional data

- Deep neural networks



REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Hill in Fog Analogy

Imagine:

You are standing on a **mountain**

Thick fog prevents seeing the valley

You want to reach the **lowest point**

What can you do?

Feel the slope under your feet

Step **downhill**

Repeat until slope becomes flat

Each step:

Lowers your height

Moves you closer to the valley

→ **This process is gradient descent**

Analogy	ML Concept
Height	Cost value
Valley	Minimum error
Slope	Gradient
Step size	Learning rate
Walking	Parameter update

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Cost Function (How Error Is Measured)

To know *how bad* our predictions are, we define a **cost function**.

The most common for regression is **Mean Squared Error (MSE)**:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

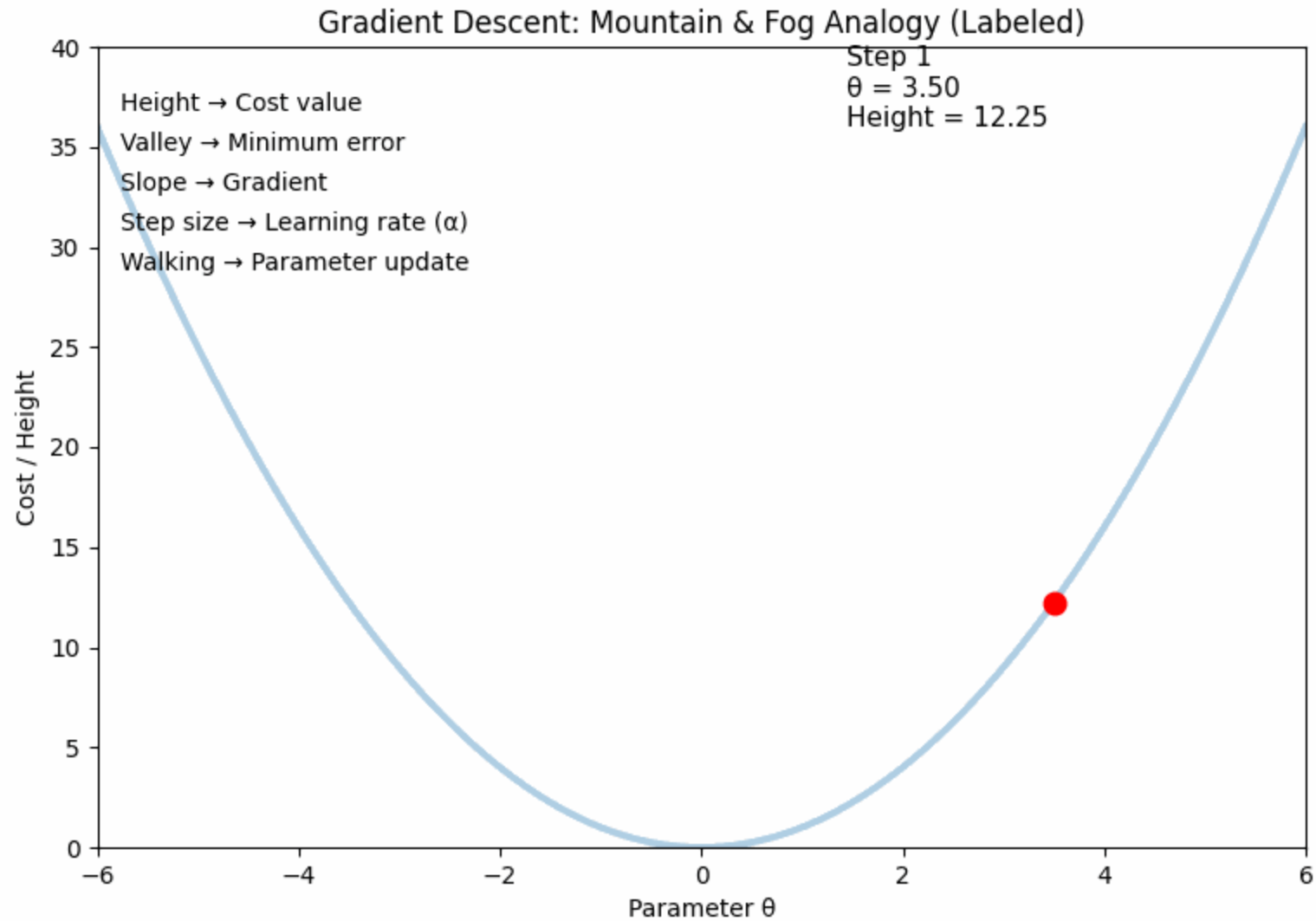
- Large difference between y and $\hat{y}$ → large cost
- Small difference → small cost
- Squaring:
- Penalizes large errors heavily
- Ensures cost is always positive

Learning Objective

$$\min J(\theta)$$

→ Adjust parameters so predictions improve.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Gradient Descent Update Rule (Step Logic)

Gradient descent updates parameters as:

$$\theta_{new} = \theta_{old} - \alpha \times \nabla J(\theta)$$

Where:

$\nabla J(\theta)$ = gradient (slope)

α = learning rate

Gradient points in direction of **steepest increase**

Minus sign moves **opposite direction**

Learning rate controls **step size**

→ Small controlled steps toward minimum

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Model

$$\hat{y} = \theta x$$

Variable	Value
x	2
y	6
Initial θ	1
Learning rate α	0.1

Step 1: Prediction

$$\hat{y} = \theta x = 1 \times 2 = 2$$

Step 2: Compute Error

$$(y - \hat{y})^2 = (6 - 2)^2 = 16$$

Large error $\rightarrow$ poor model

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Model

Variable	Value
x	2
y	6
Initial θ	1
Learning rate α	0.1

$$\hat{y} = \theta x$$

Step 3: Compute Gradient

Cost function derivative:

$$\frac{dJ}{d\theta} = 2(\theta x - y)x$$

Substitute values:

$$= 2(2 - 6)(2) = -16$$

Meaning:

- Negative slope
- Cost decreases if θ increases

LEARNING AUGMENTED

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Model

Variable	Value
x	2
y	6
Initial θ	1
Learning rate α	0.1

$$\hat{y} = \theta x$$

Step 4: Update Parameter

$$\theta = 1 - 0.1(-16) = 2.6$$

Interpretation

- θ increased in correct direction
- Prediction improves
- Cost decreases
- **Model learns**

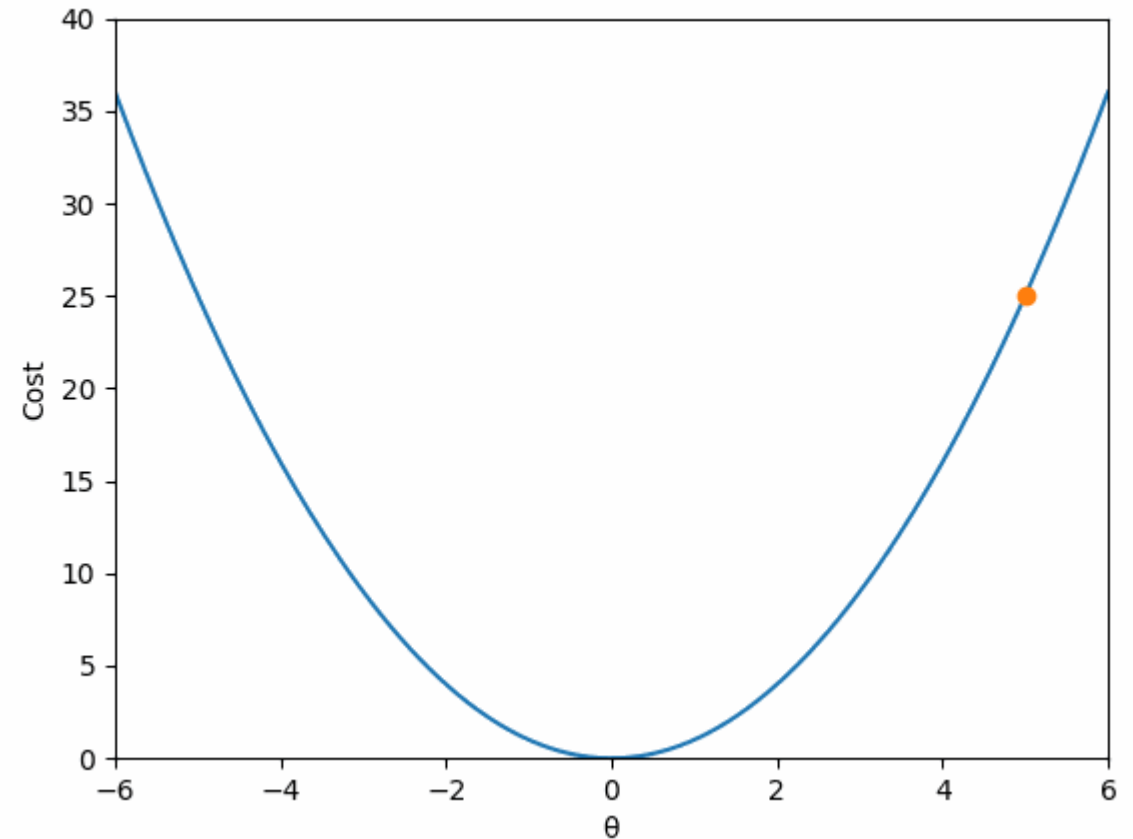
GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Learning Rate Effect

Learning rate determines **how big each step is**.

Case 1: Too Small α

- Learning is very slow
- Requires many iterations
- May stop before reaching minimum



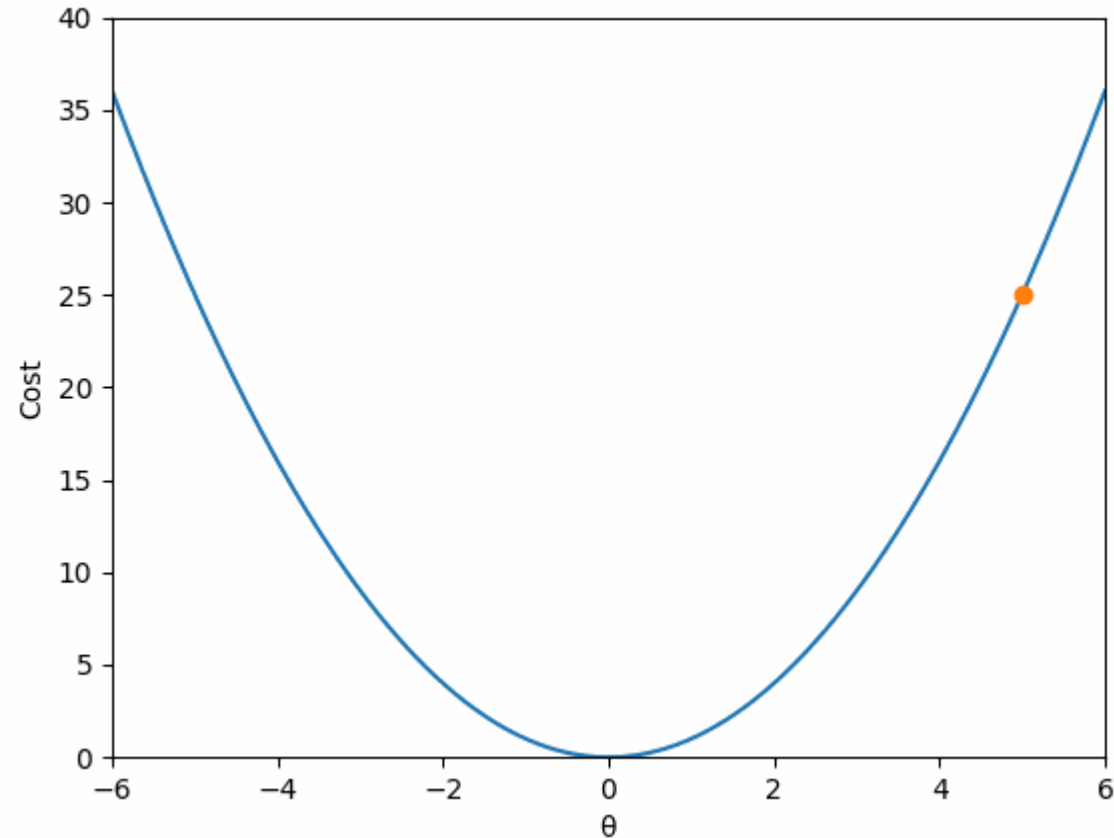
GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Learning Rate Effect (Very Important Concept)

Learning rate determines **how big each step** is.

Case 2: Too Large α

- Overshoots minimum
- Oscillates
- May diverge



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

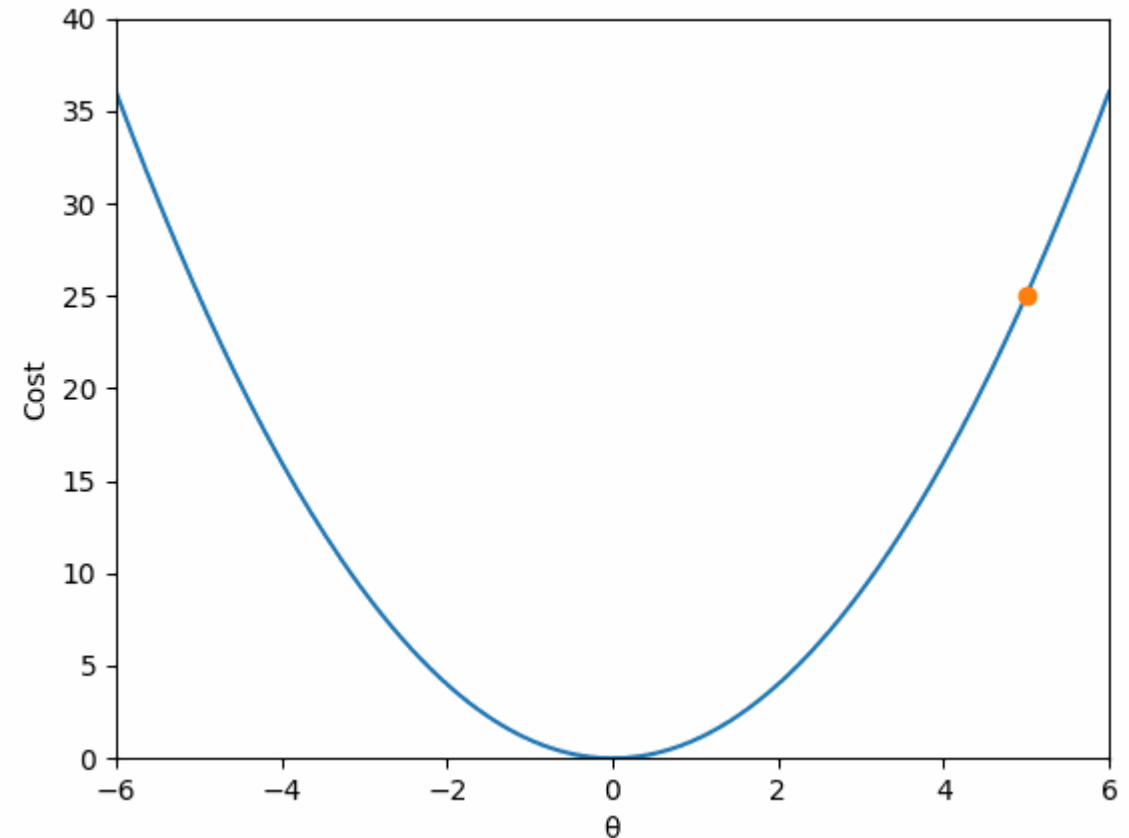
Learning Rate Effect (Very Important Concept)

Learning rate determines **how big each step** is.

Case 3: Proper α

Stable descent

Fast convergence



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Convergence

In gradient descent, **convergence** means:

The learning process has reached a point where **further updates do not significantly improve the model.**

At convergence:

The model has learned the best possible parameters

The error cannot be reduced further in a meaningful way

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Condition 1: Gradient ≈ 0

Why This Matters

The **gradient** tells us:

- The direction and rate at which the cost function changes
- Mathematically:

$$\nabla J(\theta) = 0$$

means:

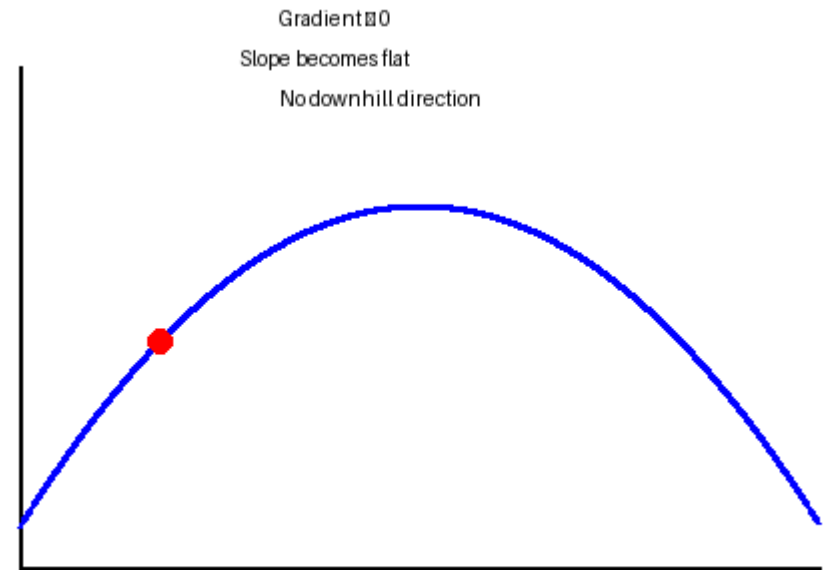
- The slope of the error curve is flat
- There is no downhill direction left

When:

$$\nabla J(\theta) \approx 0$$

It means:

- The slope is **flat**
- There is **no downhill direction**
- Any step would **not reduce cost**
→ **Learning naturally stops**



Gradient descent converges when the gradient becomes approximately zero, indicating that the cost function has reached a minimum and no further reduction in error is possible.

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Condition 2: Cost Function Stops Decreasing

What Happens During Training

Initially:

Each iteration significantly reduces error

As training progresses:

Improvements become smaller

Cost curve flattens

Mathematical View

If:

$$|J(\theta_{t+1}) - J(\theta_t)| < \epsilon$$

for a small threshold ϵ , we say:

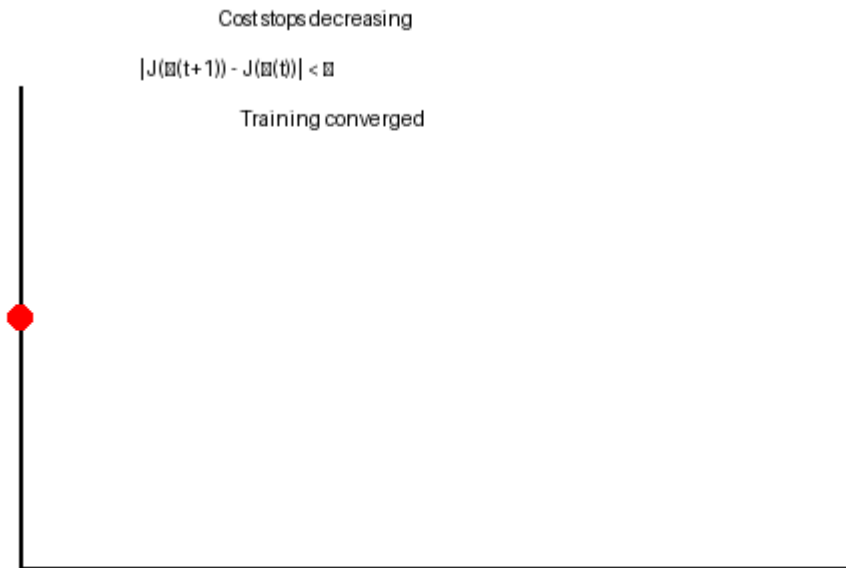
Cost has stabilized

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Condition 2: Cost Function Stops Decreasing

What Happens During Training



1. Initial phase

1. Cost drops sharply
2. Each iteration gives large improvement

2. Later phase

1. Cost curve begins to flatten
2. Improvements become smaller

3. Final phase

1. Consecutive points almost overlap
2. Cost barely changes

→ **Training has stabilized**

- Further training gives **negligible benefit**
 - Error reduction is **almost zero**
 - Continuing is **computationally wasteful**
- **Gradient descent has converged**

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Condition 3: Parameters Stabilize

What Does Parameter Stabilization Mean?

Parameters stop changing significantly:

$$|\theta_{t+1} - \theta_t| \approx 0$$

Gradient updates become very small

Learning has effectively stopped

Model has settled into optimal values

1. Early iterations

1. Large parameter jumps
2. Model is still learning fast

2. Middle iterations

1. Updates become smaller
2. Learning slows down

3. Final iterations

1. Points almost overlap
2. Parameter value stops changing

→ **Model has settled**



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

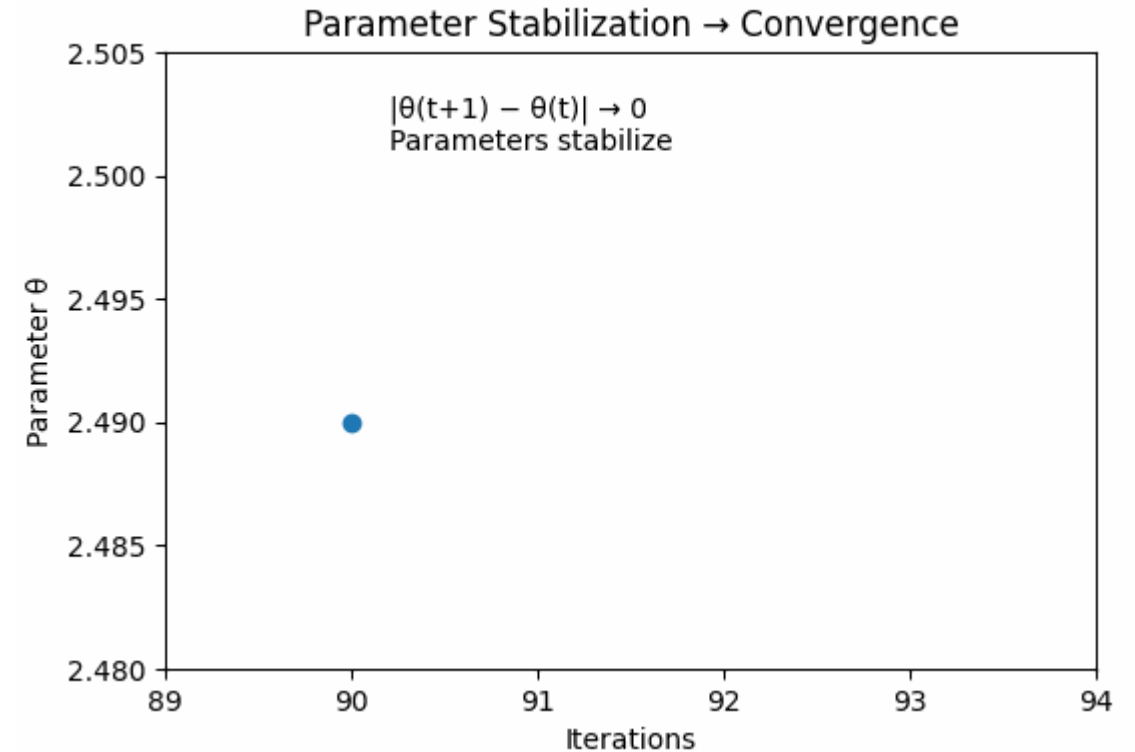
Convergence in Gradient Descent

Condition 3: Parameters Stabilize

What Does Parameter Stabilization Mean?

Iteration	θ
90	2.49
91	2.495
92	2.497
93	2.498

Changes are negligible $\rightarrow$ convergence



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Convergence in Gradient Descent

Why All Three Conditions Are Related

Condition	Meaning
Gradient ≈ 0	No direction to move
Cost stable	No improvement possible
Parameters stable	Learning completed

LEARNING
AUGMENTED

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

When Gradient Descent Fails to Converge Possible Reasons

- Learning rate too large → oscillation
- Learning rate too small → very slow
- Poor feature scaling
- Non-convex loss (local minima)

Stopping Criteria in Practice

In real implementations, training stops when:

- Maximum epochs reached **OR**
- Gradient norm $<$ threshold **OR**
- Validation error starts increasing (early stopping)

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Local vs Global Minima in Gradient Descent

1. Minimum

A **minimum** of a function is a point where the function's value is **smaller than its neighbours**, i.e., the function has reached a “low point”.

In machine learning:

We minimize the **cost function** $J(\theta)$ to reduce prediction error.

Goal: find the best **parameters** θ for the model.

Mathematically:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

where $J(\theta)$ measures how wrong our predictions are.

LEARNING
IS
AUGMENTED

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Local vs Global Minima in Gradient Descent

2. Local Minimum

Definition

A **local minimum** is a point where $J(\theta)$ is **smaller than the values of nearby points**, but not necessarily the smallest overall.

$$J(\theta_{local}) \leq J(\theta) \text{ for nearby } \theta$$

Example

Think of a **hilly landscape**:

A small dip is a **local valley**

Nearby, it's the lowest point

But there could be **deeper valleys elsewhere**

In ML, the model might settle in this local minimum if starting parameters are close.

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Local vs Global Minima in Gradient Descent

3. Global Minimum

Definition

A **global minimum** is the **absolute lowest point** of the cost function across all possible θ :

$$J(\theta_{global}) \leq J(\theta) \forall \theta$$

This is the **ideal solution**

In convex problems (like linear regression with MSE), **local minimum = global minimum**

Intuitive Example

Imagine the **lowest point of all valleys** in the landscape
Gradient descent ideally wants to reach this point

Local vs. Global Minima in Gradient Descent



GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

4. Connection to Gradient Descent

Gradient descent updates parameters:

$$\theta_{new} = \theta_{old} - \alpha \nabla J(\theta)$$

Gradient points in direction of **steepest increase**

Negative gradient → move **downhill**

Stop when gradient ≈ 0

Key Insight

Gradient descent **cannot “see” beyond local dips.**

If the function has **multiple minima**:

Gradient descent may converge to a **local minimum**, depending on **starting point**

It might **never reach global minimum**

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

5. Convex vs Non-Convex Functions

Type	Minima	Gradient Descent Behavior	Example
Convex	1 global minimum	Always finds global minimum	Linear regression, Ridge regression
Non-Convex	Multiple local minima	May get stuck in a local minimum	Neural networks, Deep learning, Polynomial regression

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

**Example
Function:**

$$J(\theta) = \theta^4 - 3\theta^3 + 2$$

Derivative:

$$J'(\theta) = 4\theta^3 - 9\theta^2$$

Solve $J'(\theta) = 0$:

$$\theta = 0, \theta = 2.25$$

$\theta = 0 \rightarrow$ local minimum

$\theta = 2.25 \rightarrow$ global minimum

If gradient descent starts near $\theta = 0$, it may get stuck there.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

GRADIENT DESCENT (HOW MODELS LEARN PARAMETERS)

Strategies to Reach Global Minimum

Multiple Random Initializations

- Run gradient descent from different starting points
- Choose best solution

Adaptive Optimizers

- Adam, RMSProp → help escape shallow local minima

Momentum

- Helps “push through” small dips

Regularization

- Smooths cost function → fewer local minima

Learning Rate Tuning

- Too small → slow convergence
- Too large → overshoot global minimum

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Why Regularization Is Needed

When we train a machine learning model:

Without constraints, the **coefficients** θ can grow very large.

Large coefficients make the model **too sensitive** to small fluctuations in the training data
→ **overfitting**.

Overfitted models perform **well on training data** but **poorly on unseen test data**.

Solution: Regularization

Adds a **penalty term** to the cost function

Discourages very large coefficients

Produces a **simpler, more generalizable model**

Intuition:

Think of a rubber band holding coefficients. If it's too loose → model grows wildly.

Regularization “tightens” the rubber band → stabilizes the model.

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Basic Idea

Instead of minimizing only the training error:

$$J(\theta) = \sum (y - \hat{y})^2$$

We **modify the objective function**:

$$\text{New Cost} = \text{Error} + \text{Penalty}$$

Error: How well the model fits training data

Penalty: How large coefficients are (controls complexity)

Effect:

Large coefficients → higher penalty → model discouraged from overfitting

Small coefficients → lower penalty → model allowed to fit patterns

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Ridge Regression (L2 Regularization)

Cost Function:

$$J(\theta) = \sum (y - \hat{y})^2 + \lambda \sum \theta^2$$

Where:

λ = regularization strength

$\sum \theta^2$ = sum of squared coefficients

Effects:

Shrinks coefficients gradually

Keeps **all features**, just reduces their impact

Smooths the model → reduces sensitivity to noise

Intuition:

Penalizes **large swings** in the model

Model becomes more “stable”

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Lasso Regression (L1 Regularization)

Cost Function:

$$J(\theta) = \sum (y - \hat{y})^2 + \lambda \sum |\theta|$$

Penalizes the **absolute value** of coefficients

Some coefficients can become **exactly zero**

Performs **feature selection automatically**

Effect:

Removes irrelevant features

Focuses on important predictors

Intuition:

Like pruning a tree: unnecessary branches (features) are cut off

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Example

Given:

$$\theta_1 = 5, \theta_2 = 0.2$$

Ridge (L2) Penalty

$$\text{Penalty} = \theta_1^2 + \theta_2^2 = 5^2 + 0.2^2 = 25.04$$

Lasso (L1) Penalty

$$\text{Penalty} = |\theta_1| + |\theta_2| = 5 + 0.2 = 5.2$$

Interpretation:

Ridge gently reduces large coefficients

Lasso can **zero out small coefficients**, performing **automatic feature selection**

LEARNING IS
AUGMENTED

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 1: Ridge Regression (L2)

Given:

Training data:

x	y
1	2
2	4
3	6

Model: $\hat{y} = \theta_1 x$ no bias for simplicity

Initial parameter: $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Regularization strength: $\lambda = 0.5$

Step 1: Compute Predictions

$$\hat{y} = \theta_1 x = 0 \cdot x = 0$$

x	y	$\hat{y}$	Error (y - $\hat{y}$)
1	2	0	2
2	4	0	4
3	6	0	6

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 1: Ridge Regression (L2)

Given:

Training data:

x	y
1	2
2	4
3	6

Model: $\hat{y} = \theta_1 x$ no bias for simplicity

Initial parameter: $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Regularization strength: $\lambda = 0.5$

Step 2: Compute Regularized Cost (Ridge)

Cost function for Ridge:

$$J(\theta_1) = \sum (y - \hat{y})^2 + \lambda \theta_1^2$$

Compute terms:

$$\sum (y - \hat{y})^2 = 2^2 + 4^2 + 6^2 = 4 + 16 + 36 = 56$$

Penalty term:

$$\lambda \theta_1^2 = 0.5 \cdot 0^2 = 0$$

Total cost:

$$J(\theta_1) = 56 + 0 = 56$$

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 1: Ridge Regression (L2)

Given:

Training data:

x	y
1	2
2	4
3	6

Model: $\hat{y} = \theta_1 x$ no bias for simplicity

Initial parameter: $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Regularization strength: $\lambda = 0.5$

Step 3: Compute Gradient

Gradient of Ridge cost:

$$\frac{dJ}{d\theta_1} = -2 \sum x_i (y_i - \theta_1 x_i) + 2\lambda\theta_1$$

Plug in values:

$$\begin{aligned} & -2[(1 \cdot 2) + (2 \cdot 4) + (3 \cdot 6)] + 2 \cdot 0.5 \cdot 0 \\ & -2[2 + 8 + 18] + 0 = -2 \cdot 28 = -56 \end{aligned}$$

Step 4: Update Parameter (Gradient Descent)

$$\begin{aligned} \theta_1^{new} &= \theta_1^{old} - \alpha \frac{dJ}{d\theta_1} \\ \theta_1^{new} &= 0 - 0.1(-56) = 5.6 \end{aligned}$$

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 1: Ridge Regression (L2)

Given:

Training data:

x	y
1	2
2	4
3	6

Model: $\hat{y} = \theta_1 x$ no bias for simplicity

Initial parameter: $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Regularization strength: $\lambda = 0.5$

Step 5: Updated Prediction

$$\hat{y} = 5.6x$$

x	y	$\hat{y}$
1	2	5.6
2	4	11.2
3	6	16.8

- Next iterations will gradually shrink θ_1 because of **L2 penalty**, avoiding extreme values.
This shows **Ridge shrinks coefficients** to control overfitting.

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 2: Lasso Regression (L1)

Given:

Training data:

x	y
1	2
2	4
3	6

Given:

Model: $\hat{y} = \theta_1 x$

Initial $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Lasso strength: $\lambda = 0.5$

Step 1: Compute Gradient (Lasso)

Gradient of Lasso:

$$\frac{dJ}{d\theta_1} = -2 \sum x_i (y_i - \theta_1 x_i) + \lambda \cdot \text{sign}(\theta_1)$$

• Here, $\text{sign}(\theta_1) = 0$ because $\theta_1 = 0$

Gradient:

$$-2[(1 \cdot 2) + (2 \cdot 4) + (3 \cdot 6)] + 0 = -56$$

Step 2: Update Parameter

$$\theta_1^{\text{new}} = 0 - 0.1(-56) = 5.6$$

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Problem 2: Lasso Regression (L1)

Given:

Training data:

x	y
1	2
2	4
3	6

Given:

Model: $\hat{y} = \theta_1 x$

Initial $\theta_1 = 0$

Learning rate: $\alpha = 0.1$

Lasso strength: $\lambda = 0.5$

Step 3: L1 Penalty Effect in Next Step

Next step gradient includes L1 term:

$$\frac{dJ}{d\theta_1} = -2 \sum x_i (y_i - \theta_1 x_i) + 0.5 \cdot \text{sign}(\theta_1)$$

$\text{sign}(5.6) = 1 \rightarrow$ L1 adds **+0.5 penalty**

New gradient = - (gradient is slightly smaller)

Update slows coefficient growth $\rightarrow$ **shrinks large coefficients gradually**

Step 4: Feature Selection Effect

If you have multiple features:

Lasso can **shrink small coefficients to zero**, removing unimportant features

Ridge **keeps all features**, just reduces magnitude

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Aspect	Ridge (L2)	Lasso (L1)
Penalty	Sum of squares $\sum \theta^2$	Sum of absolute values $\sum$
Effect on coefficients	Shrinks all gradually	Can set some to zero
Feature selection	No	Yes
Best for	Smooth models	Sparse models

- **Ridge (L2):** Penalizes squared coefficients → shrinkage, smooth model
- **Lasso (L1):** Penalizes absolute coefficients → some become zero → feature selection
- **Gradient descent** incorporates the penalty in updates → controls overfitting
- **λ (regularization strength)** controls the amount of shrinkage

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Effect of Regularization Strength (λ)

$\lambda \uparrow \rightarrow$ coefficients $\downarrow$

Small $\lambda \rightarrow$ mild regularization $\rightarrow$ minor effect

Large $\lambda \rightarrow$ strong regularization $\rightarrow$ can **underfit** (model too simple)

Rule of Thumb:

Too small $\lambda \rightarrow$ overfitting

Too large $\lambda \rightarrow$ underfitting

Optimal $\lambda \rightarrow$ best generalization

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Connection to Gradient Descent

Regularization modifies the **gradient** in gradient descent:

Without regularization:

$$\theta_{new} = \theta_{old} - \alpha \frac{\partial J}{\partial \theta}$$

With Ridge (L2):

$$\theta_{new} = \theta_{old} - \alpha \left(\frac{\partial J}{\partial \theta} + 2\lambda\theta \right)$$

With Lasso (L1):

$$\theta_{new} = \theta_{old} - \alpha \left(\frac{\partial J}{\partial \theta} + \lambda \text{sign}(\theta) \right)$$

Effect: Gradient descent now **shrinks coefficients** at each step, controlling complexity

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

REGULARIZATION (CONTROLLING MODEL COMPLEXITY)

Concept	Overfitting	Regularization
Problem	Model fits noise	Penalizes large coefficients
Effect on coefficients	Grow large	Shrink or zeroed
Model behavior	Sensitive, unstable	Smooth, generalizes
Examples	Linear regression without penalty	Ridge (L2), Lasso (L1)

Error (Residual) Assumptions in Regression Analysis

In regression, we assume that the relationship between the dependent variable Y and independent variables $X_1, X_2, \dots, X_p$ is:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi} + \epsilon_i$$

Here, ϵ_i is the **error term** or **residual**, representing the part of Y that the model cannot explain. To trust the regression results, ϵ_i must satisfy certain assumptions.

1. Zero Mean of Errors

Assumption:

$$E[\epsilon_i] = 0$$

Meaning:

On average, the errors cancel out. The model does not consistently overestimate or underestimate Y .

Why it matters:

If this is violated, the regression intercept β_0 becomes biased.

Example Violation:

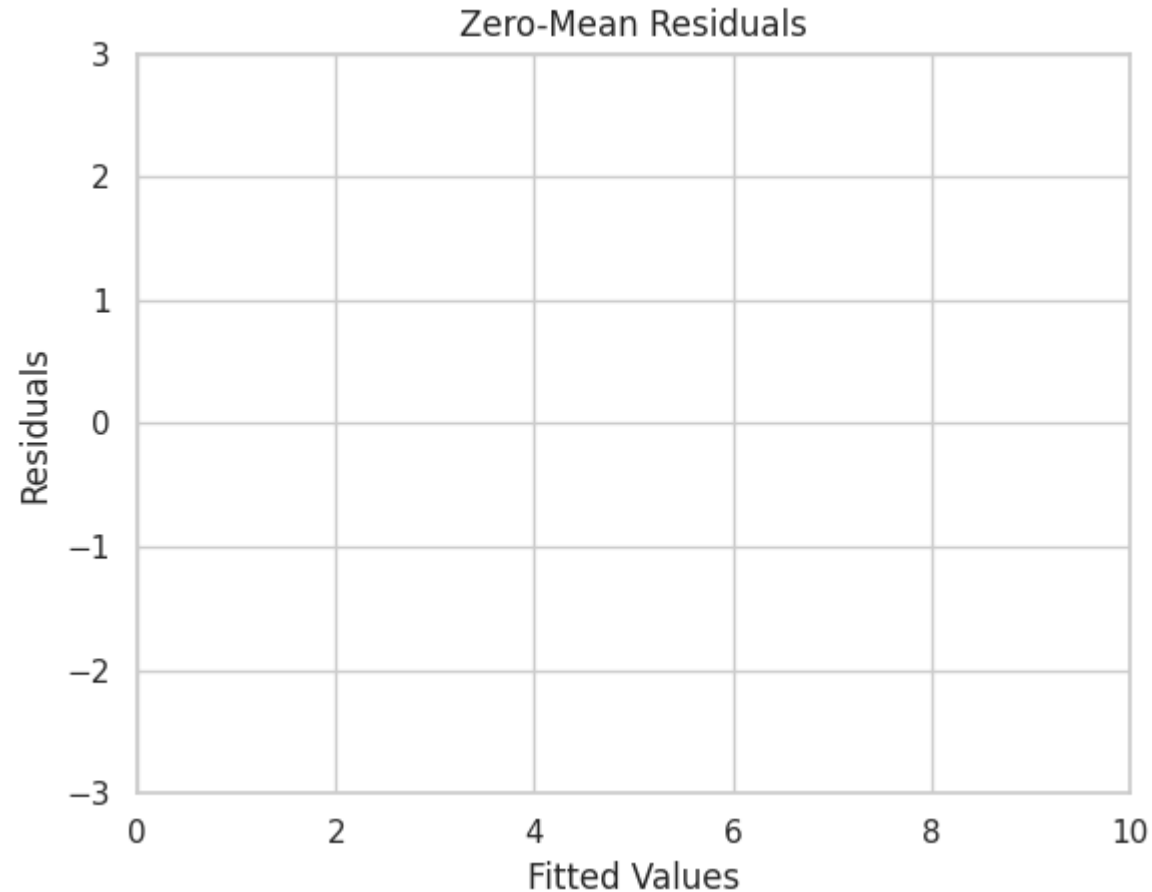
If a dataset has a systematic under-prediction at all levels of X , the mean of residuals will not be zero.

Detection:

Plot residuals; the mean should be around zero.

Error (Residual) Assumptions in Regression Analysis

1. Zero Mean of Errors



Error (Residual) Assumptions in Regression Analysis

2. Homoscedasticity (Constant Variance)

Assumption:

$$\text{Var}(\epsilon_i) = \sigma^2 \text{ for all } i$$

Meaning:

The spread of residuals is the same at all levels of the independent variables.

Why it matters:

Ensures that **standard errors** of coefficients are correct.

Violations (heteroscedasticity) make confidence intervals and p-values unreliable.

Example Violation:

Residuals grow larger as X increases (funnel-shaped pattern).

Detection:

Residuals vs. fitted plot.

Statistical tests: Breusch-Pagan test.

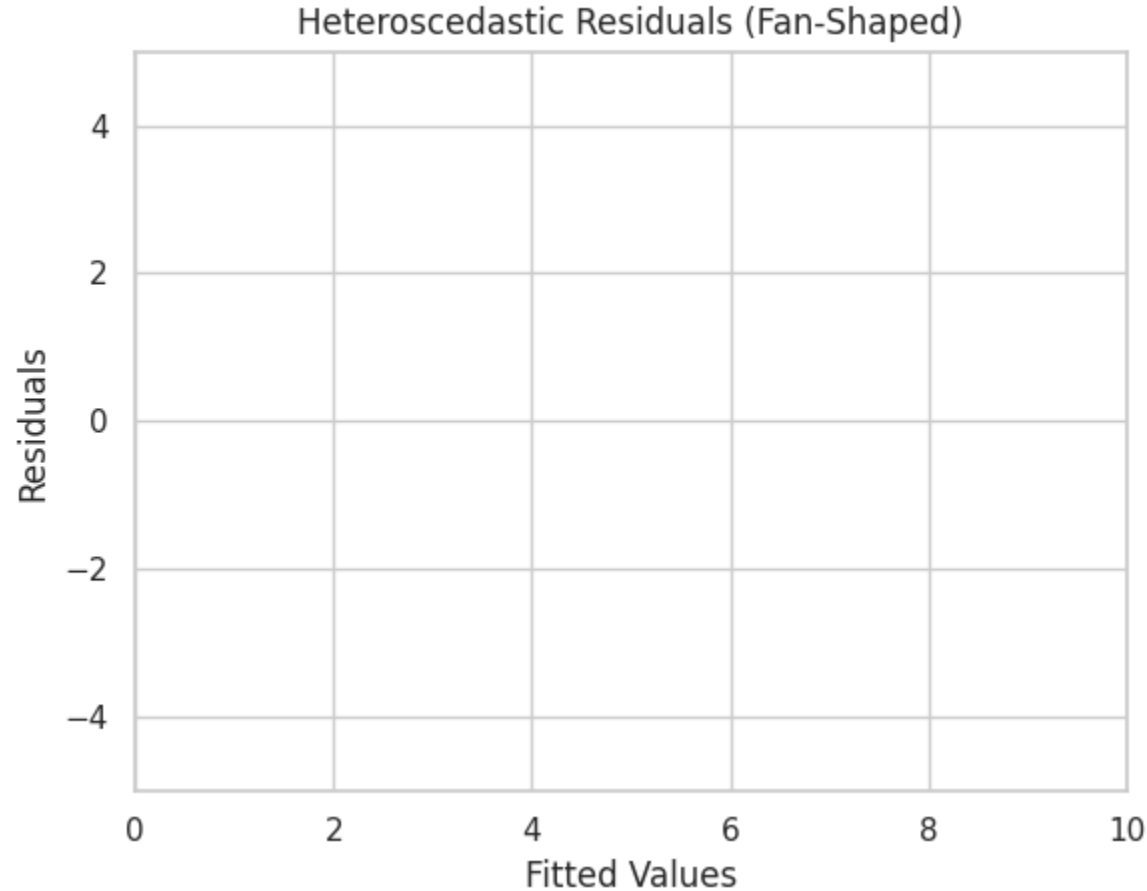
Fix:

Transform the dependent variable ($\log(Y)$), (or

Use **robust standard errors**.

Error (Residual) Assumptions in Regression Analysis

2. Homoscedasticity (Constant Variance)



Residual spread grows with fitted values → violates constant variance.

Error (Residual) Assumptions in Regression Analysis

3. No Autocorrelation (Independence)

Assumption:

$$\text{Cov}(\epsilon_i, \epsilon_j) = 0 \text{ for } i \neq j$$

Meaning:

Errors are independent; one residual does not depend on another.

Why it matters:

Common in time series data.

If violated → standard errors are biased → hypothesis tests are invalid.

Detection:

Durbin-Watson test

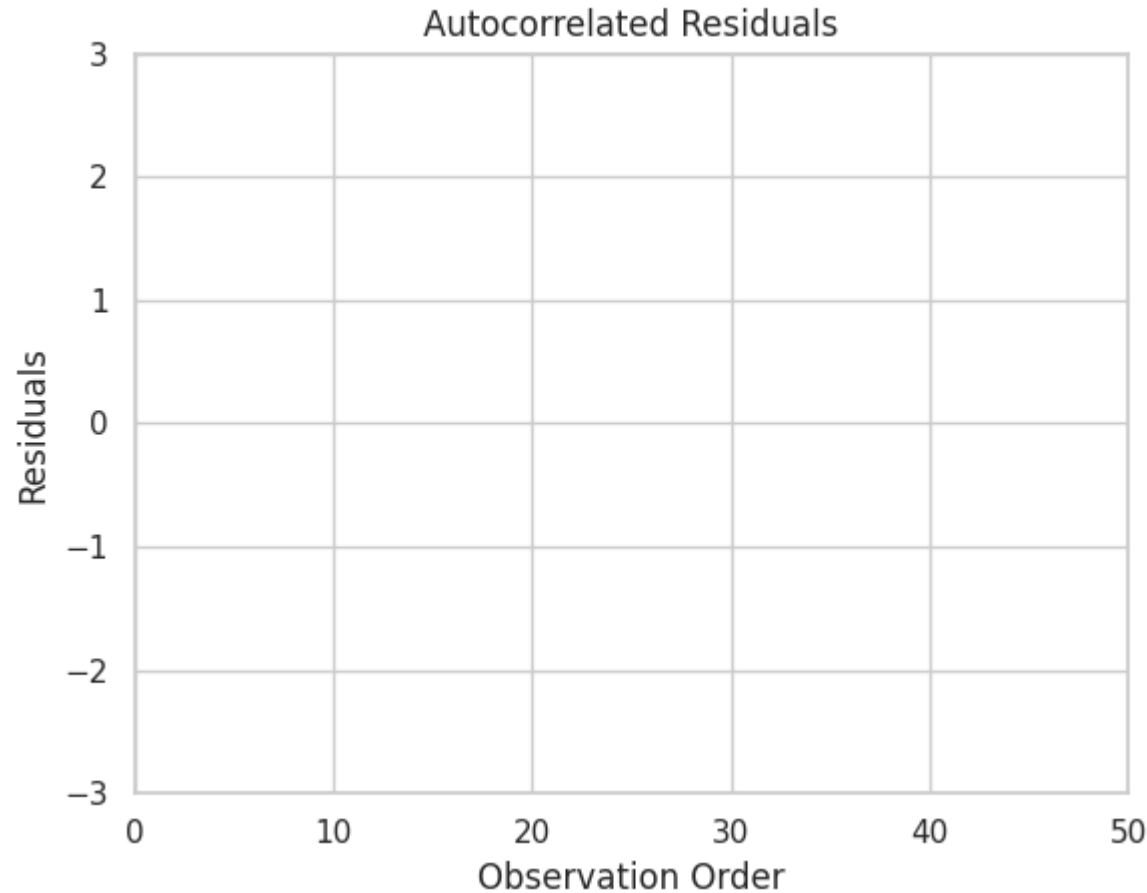
Plot residuals over time.

Fix:

Use time series models (AR, ARIMA) or add lag variables.

Error (Residual) Assumptions in Regression Analysis

3. No Autocorrelation (Independence)



Residuals are correlated with previous ones → time series problem.

Error (Residual) Assumptions in Regression Analysis

4. Normality of Errors

Assumption:

$$\epsilon_i \sim N(0, \sigma^2)$$

Meaning:

Residuals follow a normal distribution.

Why it matters:

Necessary for small-sample **t-tests and F-tests**.

If the sample is large, thanks to the **Central Limit Theorem**, this is less critical.

Detection:

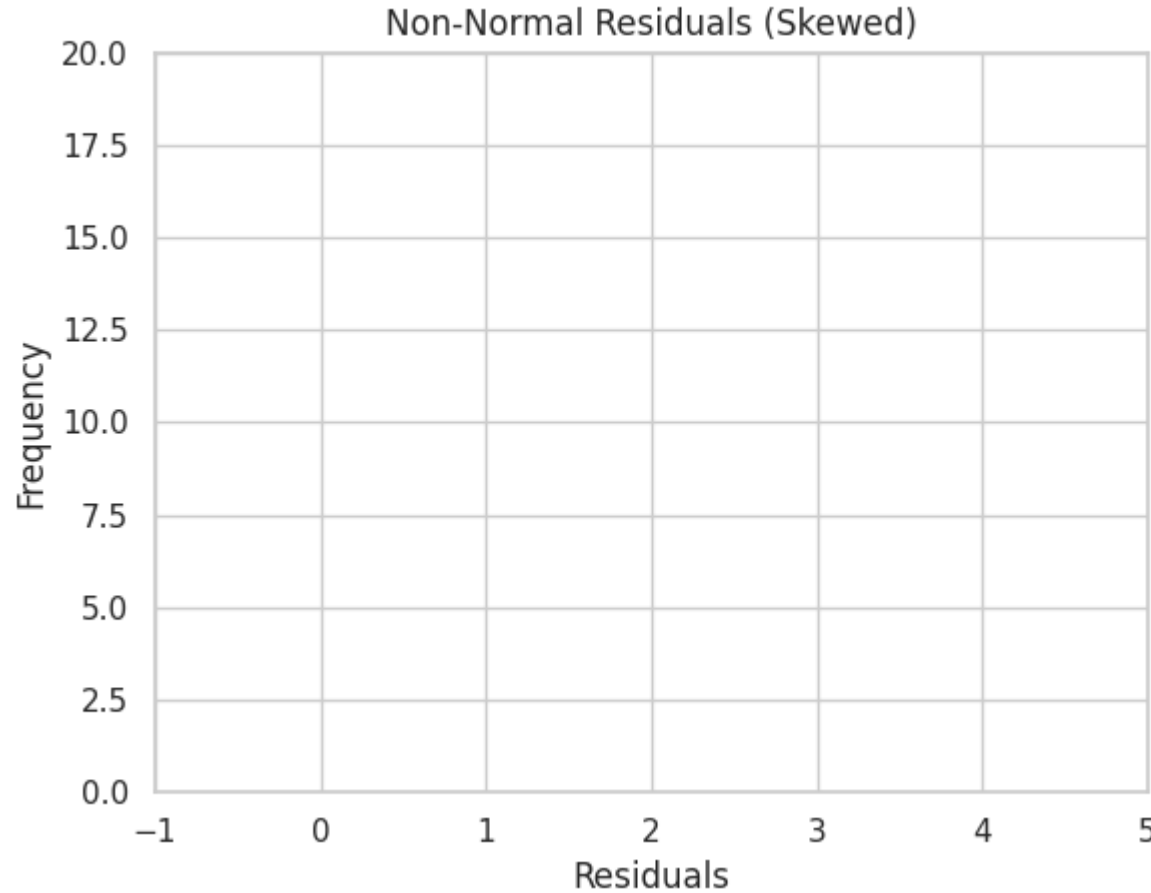
Histogram or **Q-Q plot** of residuals.

Fix:

Transform the dependent variable, e.g., log or square root.

Error (Residual) Assumptions in Regression Analysis

4. Normality of Errors



Skewed distribution $\rightarrow$ residuals not normal $\rightarrow$ t-tests invalid for small samples.

Error (Residual) Assumptions in Regression Analysis

5. No Perfect Multicollinearity

Assumption:

Independent variables should not be exact linear combinations of each other.

Why it matters:

Perfect multicollinearity makes the regression coefficients **impossible to estimate uniquely**.

Even high (but not perfect) multicollinearity inflates standard errors.

Detection:

Correlation matrix of independent variables.

Variance Inflation Factor (VIF): $VIF > 10$ indicates serious multicollinearity.

Fix:

Remove or combine correlated variables.

6. Optional: Mean of Errors Conditional on X

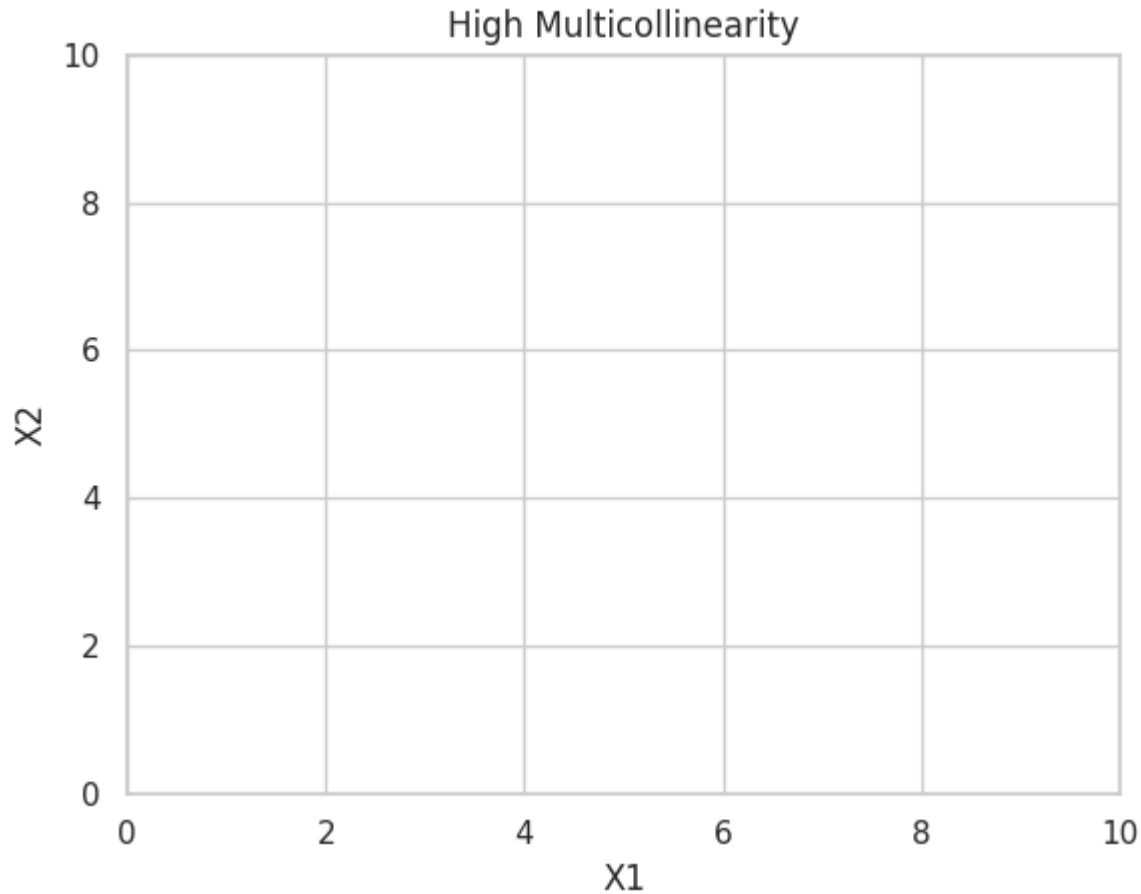
Some texts explicitly state:

$$E[\epsilon_i | X_i] = 0$$

This emphasizes that errors should not systematically depend on the independent variables.

Error (Residual) Assumptions in Regression Analysis

5. No Perfect Multicollinearity



X_1 and X_2 almost linearly dependent
→ coefficients unstable or unestimable.

REGRESSION LEARNING: FUNDAMENTAL CONCEPTS

Error (Residual) Assumptions in Regression Analysis

Assumption	Effect if Violated
Zero mean	Biased intercept
Homoscedasticity	Wrong standard errors → invalid CI and p-values
No autocorrelation	Invalid hypothesis tests, often in time series
Normality	Small-sample inference invalid
No multicollinearity	Coefficients unstable or non-estimable